Pull-Request Workflow with Agents

Last modified: 2026-08-17 19:23:17 (PDT)

The practices below apply whether you are driving an agent's work or doing the work yourself. They keep parallel sessions from colliding and keep a pull request moving toward a clean, mergeable state.

File an Issue Before Starting

When starting a **new** piece of work, go **issue-first**: before branching, editing, or opening a PR, make sure a tracking issue exists. Search the tracker first; if no open issue covers the task, **file one** (`gh issue create` / `glab issue create`), then proceed. Never jump straight into a PR without a tracking issue behind it.

The issue is the durable record of intent, scope, and "done" criteria — it gives reviewers context, lets the PR auto-close it via `Closes #N`, and keeps the work discoverable even if the PR stalls. Skip only when the task is already tracked by an open issue.

This rule settles *whether* something is tracked, not *where* it goes. An item whose deliverable is a decision rather than a diff belongs on the discussion board instead, per `choose-issue-or-discussion` — so read "file one" here as "file one in the right venue", which for actionable work is the tracker.

When the issue is a **bug report**, include a minimal reproducible example (a reprex — https://reprex.tidyverse.org/) whenever you can. A reprex is what a maintainer needs to confirm and fix the bug, and it's what they'll ask for anyway, so providing it up front saves a round trip. The `reprexes` skill helps reduce the problem to a minimal, self-contained example.

When filing an issue that contains a list of independent subissues, file each subissue as a child issue linked under the parent (GitHub sub-issues feature: `mcp__github__sub_issue_write` in remote sessions, or `gh api` with the sub-issues endpoint in local sessions).

That splitting rule has teeth, and they are worth stating: a PR's `Closes #N` closes the whole issue, including every item in it the PR never addressed. Read as tidiness,

the rule is easy to skip when the second item feels like a footnote. The actual consequence is that GitHub cannot partially close an issue, so the residual items are not deferred and not reopened — they are silently gone, and nothing in the merge, the PR, or the closed issue reports that anything was dropped.

It is worse than an ordinary lost to-do, because a closed issue is *evidence that the work was handled*. A later reader searching the tracker finds it closed and reasonably concludes every item in it was dealt with, so the loss is not merely silent but actively misleading.

So before writing `Closes #N`, re-read #N and confirm the diff covers all of it. When it doesn't, either split the remainder into its own issue first, or reference the parent with `Refs #N`, which links without closing.

- **Do:** split at filing time, or at the latest before the closing PR merges.
- **Do:** use `Refs #N` when a PR advances an issue without completing it.
- **Don't:** let `Closes #N` ride on an issue whose scope is wider than the diff.

(Morrison-Lab/ai-config#847, 2026-07-29: an issue was filed carrying a primary bug and a secondary note, and the PR fixing the first said `Closes #847`. The second item survived only because the maintainer asked about it before the merge, which is not a mechanism; it was split into #852 and shipped as #853, and both PRs merged within the following half hour. The splitting rule directly above already existed and was simply not applied when #847 was filed, which is the argument for stating its consequence rather than only its instruction.)

Claim a PR or Issue Before Working on It

Before starting a work session on a GitHub PR or issue — i.e. before fetching the branch, making edits, or invoking an automated review cycle — post a brief comment on the PR/issue so other people and any automated review bots know not to start a conflicting parallel session.

Use:

```
gh pr comment <N> --body "Working on this --- paws off until I'm done."
gh issue comment <N> --body "Working on this --- paws off until I'm done."
```

Then proceed with the work. After the session ends (PR merged, issue closed, or work otherwise paused), follow up with a closing comment so the PR/issue is unclaimed for the next person.

Skip the claim step if the most recent comment already says you are working on it. This applies to any task that will push commits to a PR branch or run iterative review loops. It does **not** apply to read-only inspection (showing a PR, checking status, explaining a diff) — those don't risk a parallel session.

This includes a PR **you opened yourself**: in repos with an active `@claude` agent (`claude.yml`), the agent can push commits to your branch on PR activity — e.g. merging `main` in — and collide with your in-flight push, so claim early to flag the branch as actively worked. (See `memories/claude-bot-workflows.md`, "(**claude?**) CI action", for the collision-recovery steps.)

When starting work from an issue, follow the claim comment with an immediate draft PR — see `pr-on-claim` for the mechanics. An open PR is a stronger "in-flight" signal than a comment alone.

Verify a mid-task "already done" claim against real PR state before trusting or redoing it. A PR you claimed and are actively driving can still gain commits from a **second, independently-running session** under the same account — a `<github-webhook-activity>` review-comment-reply event can describe work ("Addressed… Pushed in `<sha>`") that this session never did. Don't assume it's fabricated or injected, and don't reflexively redo the same fix: cross-check the PR's actual commit list (`gh pr view --json commits` / `pull_request_read get_commits`) and review threads before either (a) trusting the claim, or (b) starting the same fix yourself. If a commit with that SHA genuinely exists, authored close to when the event arrived, treat it as confirmation a live parallel session owns this PR right now — stop pushing further speculative fixes yourself, and, if genuinely in doubt, ask whether to keep driving or step back, rather than racing the other session's pushes. This gap is distinct from the initial claim check above: it's not about claiming a PR before starting, but about **re-verifying you're still the sole active driver** once work has been under way for a while — especially when you picked up the PR mid-session (e.g. by answering a diagnostic question about it) rather than through the normal claim-then-branch flow, so no fresh "paws off" check ever ran right before you started pushing. (`d-morrison/gha#286`, 2026-07-24: a webhook event delivered a review-comment reply attributed to `d-morrison` reading exactly like a Claude-authored reply, claiming a fix "Addressed… Pushed in 3fb8c5b" that this session hadn't made; verified real via `get_commits` before proceeding — a second live session, not injection.)

The git-level variant of that check: a rejected push whose remote commit is byte-for-byte what you were about to push. The section above covers a *comment* claiming work was done. Here the parallel session makes no claim at all. Your `git push` is simply rejected because it pushed first, and what it pushed is the same merge you just made. The reflex on a rejected push is to merge again, which would stack a redundant merge commit on top of an identical one.

Four reads settle it before you touch anything:

```
git rev-parse HEAD^{tree}                  # your merge's tree
git rev-parse origin/<branch>^{tree}       # theirs
git show -s --format=%P HEAD               # your merge's parents
git show -s --format=%P origin/<branch>    # its parents
```

An identical tree plus identical parents means the two merges are the same merge, so the right action is `git reset --hard origin/<branch>`.

- **Do:** compare trees and parents before deciding what a rejected push means.
- **Do:** discard your local merge with `git reset --hard origin/<branch>` once both match.
- **Don't:** re-merge reflexively on a rejected push — that is what produces the redundant merge commit.
- **Don't:** force-push over the other session's commit.

(`Morrison-Lab/ai-config#965`, 2026-07-31: `main` moved one commit, a local `git merge origin/main` was made, and the push was rejected. The remote carried `b8d2273`, a merge of the same two parents, with tree `1bda1bc`, identical to the local merge's.)

Handing off mid-task to another agent, on user request ("finish what you're doing, then relinquish holds; I'll put another agent on them"): don't just stop — leave the next agent a clean starting point. On each claimed PR/issue: (1) post a status comment on the PR itself distinguishing what's **done** from what's genuinely **not done** (the actual point of the issue, not just the side-fixes found along the way) and any blocker still open, so the next agent doesn't have to re-derive it from the diff; (2) post the closing/unclaim comment on the issue per the pattern above; (3) `unsubscribe_pr_activity` (or stop babysitting locally) so you don't keep auto-fixing a PR you no longer own; (4) stop any background watch/poll task tied to that work (e.g. a `ScheduleWakeup` or a `Monitor`/background-Bash wait) so it doesn't fire into a session that's moved on. A merge-conflict-free `git status` and a pushed branch are not enough on their own — the status comment is what makes the handoff legible. (ucdavis/bcs `gia` session, 2026-07-06: handed off PRs #310 and #311 mid-implementation this way, each blocked on the same slow `renv::restore()`.)

Keep Your Branch Synced with Main

Whenever `main` has moved ahead of a PR branch you're working on, **merge `main` into the PR branch** before the next push or review trigger. Don't wait for a conflict to surface or for someone to ask.

This fragment covers the single-branch-vs-`main` case. When orchestrating a multi-agent `ultracode` session, merges can happen at more points than that — see `ultracode-merge-conflicts` for the broader check (worktree-isolated agent branches, concurrent `parallel()` results) and the note on GitHub's mergeable indicator not evaluating custom `.gitattributes` merge drivers.

Always check for merge conflicts with main before pushing results to remote. Run this before every push, not just before triggering a review:

```
git fetch origin main
git log --oneline ..origin/main | head    # any commits? main is ahead --- merge it in
git merge origin/main
```

If the push is rejected because `main` has moved (`! [rejected]` with `(fetch first)` or `(non-fast-forward)`), fetch and merge before retrying — don't force-push.

Always do this before triggering a fresh review too, so the reviewer evaluates the PR against current `main` rather than a stale snapshot.

Don't rebase or squash-rewrite a published PR branch unless explicitly asked — a merge commit is the right move because it matches GitHub's "Update branch" button and preserves the PR history.

If the merge has conflicts, resolve them, run the project's standard pre-commit checks (render / lint / spell / tests), commit, then push. Don't push a half-resolved merge.

After merging main, re-check version parity. In R packages with a `version-check` CI job, the branch's `DESCRIPTION Version:` must *exceed* main's. A conflict-free merge can silently put them at parity — main advanced (e.g. another PR merged between when you last bumped and now). After every merge of main, compare versions:

```
git fetch origin main
git show origin/main:DESCRIPTION | grep ^Version
grep ^Version DESCRIPTION
```

If they match, bump the branch's `Version:` by one patch level before pushing.

Re-check `main` again right before the final push, not just at the start of a merge. Resolving a conflict (rerunning generators, fixing prose, updating a CHANGELOG entry) can take long enough for `main` to advance a second time. A `git fetch origin main` immediately before `git push` — after conflict resolution is done, not only before it started — catches that case; an earlier CI failure on a commit you thought was current is a symptom of skipping this second check.

A conflict-free merge does not mean derived artifacts are in sync. If your branch regenerates a generated tree (e.g. `codex-skills/`, a lockfile, rendered docs) and `main` added a new *source* input the generator consumes (a new skill, a new dependency), git merges both cleanly — but the generator never ran against the new input on your branch, so its output is missing or stale and the sync check fails on `main` after both land. After merging `main`, re-run the generator and commit the result whenever main touched the generator's inputs — don't trust the absence of conflicts. (Concretely: merge the PR that adds the new skill *first*, then sync the wrapper-regenerating branch and rerun `scripts/sync-codex-skill-wrappers.py` before merging it.)

A CI failure on a brand-new PR's very first commit (e.g. the empty claim-commit from `pr-on-claim`) is a signal to check `main`'s position before debugging the failure itself. A local checkout that sat around since before the session started can already be many commits behind `main` — the failure (a stale generated-tree check, a check `main` has since added or dropped) often isn't a real problem with your change at all, just `main` having moved. `git fetch origin main && git log --oneline ..origin/main` first; if `main` is ahead, merge it in and re-run the checks before treating the failure as something to fix in the diff. (`stack-prs` #359: an empty-commit draft PR failed `validate` on a stale `codex-skills/` generated tree, and the `require-changelog` job on a newly-added `CHANGELOG.md` requirement from PR #354 — both were `main` having advanced past a checkout that predated the session, not a defect in the new skill.)

The same staleness trap has a silent variant with no CI failure to flag it: a worktree/branch named after a PR's followup can still be based on a `main` from before that PR actually merged. A worktree directory or branch name suggesting "after PR #N" (e.g. `pr-N-followup-...`) is not proof the branch's actual base commit postdates #N's merge — it can have been created earlier and simply named for its intended purpose. Trusting that naming, then reasoning from `git show <hash>` for a commit found via `git log --all` (which lists every reachable commit across all refs, not just your branch's ancestry) can make content look present when it isn't actually in your branch yet. Verify with `git log --oneline HEAD..origin/main` or by reading the actual blob your branch would produce (`git show HEAD:<path>`, or the working tree itself before assuming what it contains), not a commit hash pulled from `--all`. If `main` has moved, merge it in before building further edits on the assumption the missing content exists. (`ai-config#637`: a worktree named `pr-636-followup-...` was cut from a `main` snapshot that predated #636's own merge; an edit referencing "the bullet above" – added by #636 – was written and committed before the bullet actually existed on the branch, caught only when `git push` reported `main has moved` and the subsequent merge produced a real conflict.)

A real conflict inside a file whose logic is also copied elsewhere (an extracted script, a doc example) needs the copy re-synced too, not just the conflicted file resolved. When a PR extracts inline logic (e.g. a workflow step's shell block) into a standalone script for testability, and `main` independently changes that same inline logic while the PR is open, resolving the merge conflict in the workflow file is not enough — the extracted script must be updated to match `main`'s new logic exactly, or the PR silently reverts `main`'s fix the moment it merges. Diff the extracted copy against `main`'s current inline version line-for-line (strip indentation, `diff`) to confirm an exact match, not just "looks about right." If the PR carries tests against the extracted copy (fixtures, unit tests), add regression coverage for whatever `main`'s change fixed — the merge is the natural moment to catch a gap the original PR's tests didn't anticipate, and to prove the new fixtures actually catch the regression (temporarily revert the fix, confirm the test fails, then restore). (gha#176: `main` landed #173's lenient verdict-matching fix to `claude-code-review.yml`'s inline fail-check logic while a PR extracting that same logic to `scripts/check-review-execution.sh` was still open; the conflict resolution

updated the script to match verbatim and added two new fixtures for #173's specific fix, verified to fail against the pre-fix logic.)

When `main` DELETES a file your branch references, resolving the marked conflict is not enough — grep the whole tree for the deleted path. Git only conflicts where both sides edited the same lines, so a merge that brings in a deletion flags the file that *used* the thing, and nothing else. Any other reference to the deleted path — a docstring citing it as precedent, a comment, a doc cross-reference — merges cleanly and silently becomes a dangling reference, because those files were never in the conflict's scope. After resolving any merge that removed a file, run `grep -rn "<deleted-path>"` across the repo and re-point or reword each hit; then distinguish live references (must be fixed) from historical citations of the removal itself (correct as-is, leave them). This is the deletion counterpart to the extracted-copy case above: there the logic moved and a copy went stale, here it vanished and the pointers went dead. (ai-config#696: `main` retired `scripts/check-new-line-breaks.py` via #703 while the PR was open. The `validate.yml` conflict was visible and resolved, but `scripts/check-memory-file-size.py`'s docstring cited the deleted script as its advisory-exit-code precedent — a file the conflict never touched, caught only by grepping for the path afterward.)

A textual conflict in a skill file can be the symptom of a conceptual duplicate, not just competing edits to the same line. When merging `main` into a branch that's authoring a new skill, if the conflict lands in a `## Relationship to other skills` section (or `main` added an entirely new skill in the same territory), that's a signal to re-run `skill-builder`'s Step 0 judgment — not just resolve the diff mechanically. Compare the new skill against whatever landed on `main`: are they the same concern (fold into one, redirect), or genuinely distinct (cross-link both directions so neither reads as an unexplained near-duplicate)? `skill-builder`'s in-flight-work scan only runs once, at the start; `main` can grow a colliding skill in the time a PR is open, so the check has to be repeated at merge time too. (PR #352's `check-info-quality` landed alongside `#344`'s independently-authored `fact-check-prose` this way — distinct enough to keep both, resolved by adding an explicit boundary in each skill's Relationship section rather than consolidating.)

The same collision can land before you write a line, and then it produces no conflict at all — just duplicated work nobody flags. The bullet above catches a duplicate at merge time, via a conflict. When `main` gains the colliding content while your change is still *planned* rather than written, there is nothing to conflict with: you write the duplicate, push it, and the review has to argue you out of content that was already redundant on arrival. So re-run the dupe check after any fetch that brings in new commits, not only at merge — a plan researched an hour ago was researched against a different `main`.

The cheap version is to read what actually arrived rather than only the count: `git log --oneline <old>..origin/main` plus `git diff --stat` over the same range, then ask whether any of it covers something still on your list. In a session that loads skills or plugins from the repo, a new one appearing in the session's own skill listing is the same signal arriving for free.

This is a *timing* gap, and it composes with the *scope* gap rather than replacing it. check-open-prs-before-duplicating covers work that is still in flight, unmerged, and therefore invisible to any check against `main`; run that one too, since a duplicate is just as wasted whether the collision has landed yet or not. Both checks share the same weakness — each runs once, at the start, and answers for the moment it ran.

Dropping the planned work is the cheap outcome, so record why in the issue and the PR body rather than deleting it silently — otherwise the next person re-proposes it. (ai-config#774, 2026-07-28: a planned `profile-before-optimising` fragment was mooted by `skills/measure-performance`, which merged via #762 during the session and covered the same two chapters — including the specific gap the fragment was meant to fill. It surfaced only because the new skill appeared in the session's skill list after a routine fast-forward; the plan had been written before it existed. Dropped before implementation, with the reasoning recorded in both the issue and the PR body, and the neighbouring fragments cross-linked to the skill instead.)

A routine merge from `main` can create the duplicate inside your own diff. The collision above lands before you write, so the duplicate is redundant on arrival. A later `main` merge is quieter: both branches were non-duplicative when they were written, and the duplicate appears only when you bring the other branch's text into yours. Git reports a clean merge because the two copies sit in different files. Diff-scoped added-line checks do not help either, because the duplicated lines already existed on one side or the other. So after merging `main` into a prose branch, run the duplicate check against the branch's full current diff and the neighbouring corpus, not only against lines added by the merge commit.

- **Do:** after a `main` merge, re-run a cross-file duplication check over the merged branch's whole prose diff.
- **Do:** treat a reviewer finding on such duplication as correct even when each copy was independently right before the merge.
- **Don't:** assume a conflict-free `main` merge preserved DRY, or that the duplicate would have appeared in an added-lines-only scan.
- **Don't:** answer by asking which branch "introduced" the duplication; the merge introduced the state that made both copies coexist.

(Morrison-Lab/ai-config#969, 2026-08-01: #969 added `shared/workflow/batch-merge-and-resolve.md` with a blockquote generalizing that an added-lines-only instrument is unsound when a defect can be introduced by deleting a line. PR #966 independently added the same generalization to `shared/workflow/sync-with-main.md` and merged after #969's branch was written. `git show 50afe818:shared/workflow/sync-with-main.md`, normalized for whitespace and markup, did not contain the phrase, so the duplication did not exist at #969's pre-merge head. The round-2 merge from `main` brought #966's copy in, and the round-3 review correctly flagged the two uncited copies.)

Two PRs that each append a new terminal numbered subsection to the same file (e.g. `### 5. ...` in a `CLAUDE.md` review-guidelines list) will conflict on merge even when neither side's content actually disagrees. This isn't an editorial clash — it's two

authors both writing to "the next number" at the same insertion point. Resolve by keeping **both** additions and renumbering sequentially from the collision point on, not by dropping either side; then grep the file for any other place that names the old numbering (a cross-reference, an index). This is also a reason `fully-clean`'s CI-green-and-review-clean verdict is a snapshot, not a mergeability guarantee — `main` can pick up its own append in the same spot after your last review round, so a PR can go from "reviewed clean" to "needs a merge conflict resolved" with no defect in its own diff. Before reporting a PR ready to merge, re-check with `git fetch origin main` plus the `git merge-tree` command from `resolve-conflicts`, not just a cached `mergeable` flag or an earlier green CI run. (gha#211: `main` merged #209's own new `### 5. Check for AI-generated prose tells` subsection between this PR's clean review and its actual merge — `git merge-tree` surfaced a real conflict that neither PR's own CI nor review status had flagged, since neither had rerun since `main` advanced.)

After merging a PR that extracts an inline block into a reusable unit (a composite action, a shared script/function), check other open PRs that still edit that same inline block — your merge just broke their textual diff, even though their intended change is usually trivial to re-apply to the new location. This is the mirror image of the case above: there, you're the one resyncing after `main` moved a copy of your logic; here, *you* are the one who moved the logic, so the burden of noticing and fixing the resulting conflict falls on you, not on the sibling PR's author waiting to hit it. Don't wait for that PR's own merge/CI to surface the conflict — check every open PR touching the same file right after your extraction merges: `git merge-tree "$(git merge-base origin/main origin/<sibling-branch>)" origin/main origin/<sibling-branch>` (or `gh pr diff <N>` against the new `main`) shows whether it still applies cleanly. Re-apply the sibling PR's actual semantic change (not a mechanical `--theirs`) to the new location, verify with a direct diff that the extracted unit now differs from `main` by exactly that PR's intended change and nothing else, then push to their branch and flag what you did in a PR comment. (gha#201 extracted `claude-code-review.yml`'s `claude_args` block into a new `run-claude-review-attempt` composite action to support a retry; gha#202, open in parallel, edited that same inline block to allowlist `WebFetch`/ `Bash(curl:*)`. Proactively rebasing #202 and re-applying its allowlist change to the new composite action — rather than leaving its author to discover a conflict — let it merge within the hour instead of stalling.)

That "push to their branch" is scoped by standing, not only by cause. gha#201/#202 were CI workflow files in a repo the author drove, where a push saves the sibling's author a round and risks nothing they were relying on. The same push onto a branch you do not own — a colleague's active work, and most sharply a release branch carrying an out-of-band process — can disrupt something a comment would not. There, name the extraction, the deletion, or the rename and where the content went in a PR comment, and leave the push to whoever owns the branch. Causing the conflict obliges you to *surface* it. It does not by itself license editing someone else's branch. See `batch-merge-and-resolve`, "A conflict your sweep found is not a conflict your merge caused", for the attribution step that says which conflicts are yours in the first place.

An add/add conflict on a *shared config file* usually means two PRs independently fixed the same root cause — reconcile the reasoning, don't just pick a side. This generalizes the skill-file case above beyond skills: a repo-wide CI/lint/build config fix (a new tool config file, a workflow tweak) is exactly the kind of change multiple sessions or bots are likely to attempt in parallel once a check starts failing on `main` for everyone. When the conflict is a whole-file add/add (not just competing edits to an existing file), read both sides' reasoning — code comments, commit messages, the PR discussion — before resolving; usually one side's explanation is more complete (covers a case the other missed, cites the tool's actual constraint) and should win outright rather than mechanically merging fragments of both. Re-diff the PR against `origin/main` after resolving to confirm the PR's remaining changes are its own original scope, not a reintroduction of what the other, now-merged PR already added. (`d-morrison/altdoc#7` vs `#18`: both independently added a `jarl.toml` excluding the same fixture directory for the same `jarl-check` failure; `#18` merged first, `#7`'s merge conflicted on the new file, resolved by keeping `#18`'s more detailed comment and re-confirming `#7`'s diff against `main` was back down to just its own four files. This same "append-collision" pattern struck a third time one insertion point over: this bullet and the two above it were each added by independent PRs landing in quick succession, all appending after the same "PR #352's `check-info-quality`…" paragraph — resolved, per the guidance above, by keeping all three rather than picking one.)

A `dirty` `mergeable_state` on a bot-opened PR can mean a sibling PR already closed the same issue, not just that `main` drifted. An issue-triggered `@claude` workflow can fire twice on the same issue in quick succession (a duplicate dispatch, or two people independently routing the same request), producing two independent PRs that both fully resolve it — including adding the identical new file. The second PR's merge conflict is an add/add on that new file, and it looks like ordinary main-drift, but treating it that way and mechanically resolving in favor of "ours" silently reintroduces a duplicate the other PR's merge already published. Before resolving, check the PR's linked issue for **other** cross-referenced PRs/closing events — if one already merged and closed it, diff the conflicting file against `main`: if it's the sibling PR's already-published version, keep `main`'s content and keep only this PR's genuinely distinct remainder (a piece the sibling PR never did), rather than re-adding a second copy. (`ai-config#501`: issue #500 was independently resolved twice — `#502` merged first, adding `shared/writing/math-derivation-steps.md` and closing #500, but never wiring it into `CLAUDE.md`; `#501` added a second copy of the same fragment plus the missing `CLAUDE.md` wiring. Resolved by keeping `main`'s published fragment and `#501`'s wiring, turning a `dirty` merge into a clean `+8/-0` diff.)

The same parallel resolution can be a whole-file split, and then files can vanish from your diff with no deletion hunk to read. The add/add and duplicate-issue cases above both say to keep `main` when a sibling PR already published the same new file. The split case adds a second check, because resolving the one conflict can also make other files disappear from your PR's diff entirely. Those files look harmlessly gone, and there is no deleted line for `ardi`'s pre-push deletion sweep to inspect. Two causes are indistinguishable from the final diff alone: `main` absorbed your cross-reference edit, or the merge dropped your work. So verify each

vanished file against the pre-merge head before calling the collapse correct. For each file that left the diff, compare the original head against the merge-base to recover what your branch intended, then confirm current `main` now carries that same change. Only after that per-file check is it safe to treat the smaller diff as a successful conflict resolution rather than as lost work.

- **Do:** save or read the original pre-merge head, list the files that left the PR diff after the merge, and verify each one's intended change is already on `main`.
- **Do:** keep `main`'s version for the overlapping split file when the sibling PR has already published the same refactor, then carry forward only this PR's distinct remainder.
- **Don't:** infer that a vanished file was safely absorbed merely because the final diff got smaller.
- **Don't:** rely on the deleted-lines sweep for this case; content that left the diff has no deletion hunk for that sweep to show.

(Morrison-Lab/ai-config#966, merging `origin/main` after #973 landed as `ea11bc9a`, hit `CONFLICT (add/add)` on `memories/github-mcp-tools.md`. Both branches had split that file out of `memories/github.md`; the two split versions were byte-identical apart from #966's own 49-line addition, so keeping `origin/main:memories/github-mcp-tools.md` was correct. The final PR diff collapsed from 13 files, 1093 insertions, and 661 deletions to 8 files, 429 insertions, and 6 deletions. Five files disappeared entirely: `memories/claude-bot-workflows.md`, `memories/claude-code.md`, `shared/workflow/efficient-pr-babysitting.md`, `shared/workflow/fully-clean.md`, and `skills/purge-hallucinations/SKILL.md`. Each had contained only a cross-reference repointing from `memories/github.md` to `memories/github-mcp-tools.md`, and `git show origin/main:<file> | grep -c github-mcp-tools` returned `1` for each.)

When the whole PR is superseded, not just one file, the conflict is telling you to close it rather than resolve it. The two cases above keep `main`'s version of a file a sibling PR already published, and carry forward the current PR's distinct remainder. The remainder can be empty. When a `main`-merge conflict pits *every* added line of an idle PR against a better-formatted copy already on `main` — a sibling PR having landed the same content — resolving toward `main` leaves nothing, and the PR's own prior review findings are moot. Confirm by grepping `origin/main` for the PR's distinctive added phrases before resolving anything: all present means superseded. The right action is then to recommend closing the PR, since its content is preserved on `main`, not to push an empty diff to a clean verdict. For an ARDIA sweep this is a terminal state of its own — see `ardia`'s `Superseded`, which also gives the up-front check that catches it before rounds are spent. (Morrison-Lab/ai-config#1188, 2026-08-06: an idle PR with a "Needs more work" verdict, driven toward clean, revealed on merging `origin/main` that all four of its `memories/preferences.md` bullets were already there in corrected form, landed by the already-merged #1189; resolving toward `main` would have left an empty diff, so the PR was superseded and the correct action was closure.)

A merge into a growing numbered list (e.g. `gha`'s `CLAUDE.md` "Code review guidelines"

section) can produce zero blank lines between two adjacent headings even with no textual conflict — lint catches it, git doesn't. When a section is a hotspot several PRs independently append items to (each PR adding its own `### N.` block at the end), a clean three-way merge can still splice one PR's closing line directly against the next PR's heading with no blank line between them — this doesn't produce a `<<<<<<<` conflict marker (git resolves it as a straightforward insertion), so it's easy to push without noticing. `markdownlint`'s MD022 (blanks-around-headings) is what actually catches it, as a CI failure with no proximate code change to explain it. Re-run the repo's markdown lint (or at minimum re-read the diff around every `### N.` boundary you didn't personally write) after any merge that touches a shared growing list, not just after a merge with conflicts. (gha#208: an out-of-band merge from `main` — done by a different session, not the one that opened the PR — landed a new item 7 directly against the PR's own item 6 with no blank line; `lint-markdown`'s MD022 failed with no conflict marker anywhere in the diff to point at.)

The same splice happens to LIST ITEMS, and there `markdownlint` most likely does NOT catch it — so nothing turns red at all. The case above is a heading spliced against preceding text, which MD022 decides. The changelog case is a *bullet* spliced onto the previous item's continuation line:

```
  `data-raw/precompute-true-effects-chunk.R` (#429).
* The `docs` workflow's "Build site" step no longer times out intermittently.
```

That is a valid **tight** list item, so a list in which every other entry is blank-line separated silently starts mixing tight and loose items and renders inconsistently. `markdownlint`'s blanks-around-lists rule governs the boundaries *of* a list, not the gaps *between* its items, and no default rule enforces consistent looseness within one list — so unlike the heading case, CI stays green and only a human reading the merged section notices.

Check it mechanically instead of by eye; one line decides it:

```
awk 'prev !~ /^[[:space:]]*$/ && /^[*+\-] / {print FILENAME":"NR": "$0} {prev=$0}' NEWS.md
```

Use `[[:space:]]*` rather than a bare `/^$/` — a whitespace-only preceding line is not a violation and produces false positives. The pattern `[*+\-]` covers all three common Markdown unordered-list markers; `^\*` alone would miss `-` and `+` bullets.

Two consequences. Run this after any merge into a changelog or other growing bulleted list, alongside the heading check above. And note that a `merge=union` driver on such a file (see configure-gitattributes) *increases* the rate of this defect, since union resolves an append collision by keeping both sides with no conflict to review — so confirm a detector is wired into CI before enabling one, not after. (ucdavis/bcs#422/#430, 2026-07-26: a clean three-way merge spliced one PR's `## Bug fixes` bullet against another's; the check above then found

four pre-existing instances in the same `NEWS.md`, in a repo that had no Markdown linting at all.)

Run that check as a whole-file count, and compare it before and after — scoping it to the lines you added cannot see this defect at all. The check above is the right instrument; the natural way to apply it is the wrong one. Having found the file's spliced bullets, the obvious next question is which of them are yours, and the obvious way to answer is to intersect them with the lines the branch added. That question is unanswerable, because the defect is a **deleted blank line** before a bullet that was already there. The bullet is *context* in the diff, never an addition, so the intersection is empty by construction and the check reports a confident zero.

Note how this differs from the scope failures elsewhere in this corpus, where a check's **inputs** were too narrow — a glob, a missing flag, a two-dot range. Here the inputs were right and the **question** was wrong, which no widening fixes. And it fails in the direction that reads as an all-clear, on the one file a reviewer will not re-derive.

The sound form is a count delta over the whole file, which needs no judgment about ownership and no diff at all:

```
git show origin/main:NEWS.md   | awk '...' | wc -l   # before
awk '...' NEWS.md              | wc -l               # after
```

A merge must not increase the count. That is an `algorithmatize-checks` instrument in the strict sense — two integers decide it — and it holds whoever authored the surrounding lines.

Generalize past changelogs, because the property is about the defect rather than the file: **when a defect can be introduced by deleting a line, any instrument keyed on added lines is unsound.** Ask instead whether a whole-file measurement got worse. The version-parity rule above is the same shape — a conflict-free merge leaves the branch at parity with `main`, `version-check` goes red, and there is nothing in the diff to point at — which is why that rule is a direct comparison of two `DESCRIPTION` versions rather than a diff inspection.

The shared trigger is the practical part: **a conflict-free merge is exactly when nothing prompts anyone to look.** Both defects arrive through one, both are invisible to diff-scoped checking, and the merge reports success.

- **Do:** measure the whole file before and after a merge, and treat any increase as the merge's fault regardless of who wrote the lines.
- **Do:** ask, of every diff-scoped check, whether the defect it targets could be caused by a deletion — and replace it with a count if so.
- **Don't:** intersect a whole-file finding with the branch's added lines to decide ownership; for a deletion-caused defect that always returns zero.
- **Don't:** read a conflict-free merge as a merge that changed nothing beyond what the diff shows.

(`ucdavis/bcs#534`, 2026-07-31: merging `main` spliced its `NEWS.md` bullet directly beneath the branch's own with no blank line between. markdownlint stayed green, since `blanks-around-lists` governs a list's boundaries rather than the gaps between its items. A pre-push check asking which flagged bullets were among the branch's added lines returned 0 — `git diff -U0 origin/main...1b74899d -- NEWS.md | grep -c '^+\* Say that the'` — and that zero was reported to the user as "none of these are mine", which was false. The count delta settles it: 14 spliced bullets on `origin/main`, 15 at `1b74899d` after the merge, 14 again at `9f5dab34` after the fix. Evidence filed on ucdavis/bcs#437; the `merge=union` driver proposed in ucdavis/bcs#438 would raise this defect's rate, so it is explicitly blocked on the detector landing first.)

A commit claiming "I've pulled main and resolved the merge conflicts" can be lying — verify it actually merged before trusting the claim. A genuine conflict-resolution commit is a merge commit (two parents); a commit that just hand-edits files to *look* resolved, without running a real `git merge`, is an ordinary single-parent commit — and it never actually incorporates whatever new state of `main` prompted the "resolve conflicts" request in the first place. This is easy to miss because GitHub's own `mergeable`/`mergeStateStatus` fields don't distinguish the two: both look identical from the PR page until you check the commit graph. Verify with `git show -s --format="%P" <commit>` — one hash means no real merge happened, regardless of what the commit message says. If a branch needed conflict resolution more than once and each attempt claimed success but the PR still shows `CONFLICTING`, check every "resolved conflicts" commit in its history this way before trying yet another resolution attempt on top of a foundation that was never actually re-merged. (`Lacaedemon/sparta#1070`, 2026-07-27: an automated PR-authoring agent pushed two consecutive commits both titled "Resolve merge conflicts", each claiming to have pulled `main`; both were single-parent commits that never touched `main`'s actual current state, so the PR kept showing `CONFLICTING` no matter how many times the agent "fixed" it. A real `git merge origin/main` — the first one actually run against this branch in three attempts — surfaced the genuine conflicts and resolved them for good.)

Driving a Pull Request to Clean

Whenever you are working a PR/MR, run the full **ARDI** loop by default, without being asked: **A**ddress every flagged item, **R**ebut findings that are wrong, **D**efer out-of-scope items to tracked issues, then **I**terate with a fresh review — repeating until the latest review is **fully clean**. Don't stop at "review-clean, just needs approval" and hand triage back; keep the cycle going until it's genuinely clean.

Worked-example case records for the rules below live in `ardi.cases.md`, moved out of the auto-loaded context.

Continuously monitor every PR/MR you are actively working until it reaches that terminal state. At every periodic check-in, and again after any push or base-branch

advance, query the current head for all three surfaces: mergeability (including conflicts), every CI workflow/check run, and both formal reviews and top-level/inline review comments. A conflict, CI failure, or newly posted finding is ARDI work immediately — sync and resolve the conflict, investigate and fix or track the CI failure, or disposition the finding — not merely a status item to hand back to the user. Keep polling while a review or check is in progress; do not call the PR clean from an earlier head or from green CI without a current-head review verdict. Pushing fixes for a finding-bearing review starts a new review cycle: the ARDI loop is NOT finished when you push fixes or post an ARD summary. You must wait for the fresh review run evaluating your latest pushed commit to post, fetch and parse that review, and confirm it is clean before declaring the loop finished.

That wait is conditional on a run having been scheduled, and on some repos a push schedules nothing. A review workflow whose `on:` block carries no push-based trigger — `workflow_dispatch` and `issue_comment` only, which is how a repo disables automatic review on PR activity — fires nothing when you push, so the run you are told to wait for will never exist and the poll cannot terminate. The obligation is then discharged by **dispatching**, not by waiting, and it recurs on **every** push rather than once at PR-open time: `gh workflow run <review-workflow>.yml -R <owner>/<repo> -f pr_number=<N>`, taking the input's name from that workflow's own file. Read the `on:` block once per repo, the first time you push to a PR there, and record which class it is. This is the shape most likely to be missed while everything looks healthy, because CI still goes green on each push, and watching CI to green feels like watching the PR — so the loop closes on "checks passed" while the last verdict on file dates from an earlier head. `check-pr-fully-clean.py` returning non-zero for "no review at this HEAD SHA" on such a repo means *dispatch now*, not *poll longer*. `pr-on-claim` covers the PR-open and draft-to-ready end of this. The increment here is that each subsequent push owes its own dispatch.

- **Do:** read the review workflow's `on:` block before the first push to a PR in an unfamiliar repo, and dispatch explicitly after every push when it carries no push-based trigger.
- **Do:** treat a non-zero `check-pr-fully-clean.py` on a dispatch-only repo as a prompt to dispatch.
- **Don't:** read green CI at the current head as evidence a review is in flight — on a dispatch-only repo that is the steady state, not a transient one.
- **Don't:** let a verdict from an earlier head stand because the repo's trigger class was already known. Knowing it is not the same as acting on it each round.

NEVER use background tasks, async sleep commands, or schedule timers for ARDI status polling. Always execute `python3 scripts/check-pr-fully-clean.py <pr>` synchronously in the foreground turn. This applies transitively to PR-driving workflows such as `gi`, `gii`, and `ardia`; only monitor PRs the session owns or has explicitly claimed, so the rule does not authorize changing someone else's work.

The loop's terminal action is to **report the PR ready, not to merge it**. Merging is human-gated — it happens only on an explicit human "merge it" (the `merge-it` skill), never

as a step ARDI takes on its own. So when you carry a PR across a `ScheduleWakeup` or `/loop` wait, **never** bake a self-merge directive like "if clean and CI green, merge it" into the wakeup/loop prompt: a scheduled prompt fires back as a user-role turn, so a self-authored "merge it" only *looks* like human approval (and Claude Code's auto-mode classifier will rightly deny it as a self-authored merge). Drive to fully clean, report ready, and leave the merge — and any other destructive one-off, e.g. a `gh workflow run` that force-pushes — for explicit human authorization.

Because the loop ends there, **the clean verdict is also where `ums` runs** — don't hold the pass for the merge, which is on the human's clock rather than this session's and may land after a `/clear` or not at all. See `CLAUDE.md`'s "Run UMS proactively, as learnings accumulate"; the merge-time pass in `post-merge` then only has to cover what the merge itself taught.

The one exception: if the human has explicitly granted the `mwc` (merge-when-confident) session permission, that grant is a live human instruction, not a self-authored one, so baking a self-merge step into a wakeup/loop prompt is fine for the rest of that session. See `mwc` for the grant's scope and limits.

In the **clear-all family** (`ardia`, `gia`, `gii`, `gip`), "report ready, don't merge" gates only the merge — it does **not** pause the sweep. A clean-but-unmerged PR is not a stop; move to the next item, and stack it when it isn't naturally independent of that PR. See `stack-dont-pause`.

The same gate does not pause the loop *within* a single PR either, and that is the harder half to see. The paragraph above says the merge gate does not stop you moving to the *next* PR. This says it does not stop you working *this* one. An authorization gate attaches to a specific **action** — the merge, a force-push, a destructive one-off — never to the PR as a whole. Everything else the loop already mandates stays pre-authorized on that PR: syncing with `main`, resolving a conflict, pushing a fix, re-dispatching a review, resolving threads, reporting the result. `CLAUDE.md`'s "Watch and ARDI every PR you touch — don't ask first" states the standing yes; this names the boundary it stops at.

Conflict resolution in particular is not merely permitted but **owed**, per the continuous-monitoring paragraph at the top of this fragment: a conflict "is ARDI work immediately — sync and resolve the conflict … not merely a status item to hand back to the user". Handing it back is the named anti-pattern rather than a cautious reading of the merge gate.

What makes this hard to catch from the inside is that it is not laziness or evasion. The gate being over-applied is a **real** gate, correctly identified, and usually one you have already invoked several times on that same PR for the action it genuinely covers. Having rightly refused to merge, refusing to touch it at all reads as consistency rather than as a second and different refusal. So the lesson is not "be more proactive" — diligence was never the missing input. It is that a gate has a scope, and the scope is the action.

The tell is lexical, and it sits in your own outgoing message: a RECOMMENDATION or question whose proposed action is ordinary ARDI work. If the sentence

you are about to write asks permission to do something the loop already requires, that is the error. Do it, and report in the past tense.

This bites hardest on a PR that **cannot** reach a clean verdict — no external reviewer will answer at any head, so nothing about it feels routine and the whole PR starts to read as gated. Drive it to whatever state it *can* reach: merged with current `main`, green on every check that runs, threads resolved, self-review posted. Then report it as blocked on the specific thing it is actually blocked on, rather than leaving it dirty because the terminal step is unavailable.

- **Do:** resolve conflicts, sync, push fixes, and re-dispatch reviews on a PR whose merge you are correctly withholding, and report those in the past tense.
- **Do:** name the one action that is gated, so "blocked" stays a claim about a step rather than about the PR.
- **Don't:** generalize a withheld merge into withholding the rest of the loop as though one authorization covered both.
- **Don't:** write a recommendation proposing work ARDI already mandates — a request to do the required thing is the error, not a courtesy.

See `ardi.cases.md`, "A merge gate is not a work gate".

Self-review against the project's own stated conventions before every push, not just the first — and don't just re-read the criteria, actually run the applicable review skills against your own diff and iterate on what they find, the same ARD cycle you'd run against an external reviewer's findings. Don't treat the review bot as the mechanism that discovers a project's documented conventions — self-apply them first. When a project's own `CLAUDE.md` (or equivalent agent doc) already states specific criteria — a DRY/no-duplication rule, a doc-sync checklist for a new input, a changelog-category rule, a citation requirement, a "new logic needs test coverage" norm, a prose-quality check like `fact-check-prose`, `fix-forward-references`, or `detect-informal-definitions` — a first-pass implementation checked only against feature correctness forces the review loop to spend a round re-deriving what the project's own docs already said. Before every push, re-read the project's own stated review criteria and actually invoke the review skills/checks it names against the diff (not just recall them from memory), the same way an external reviewer would apply them. Address every finding your own self-review surfaces — fix, rebut, or defer, exactly like the ARD step above — before the push goes out; a self-review that finds issues and pushes anyway has only moved the round to the external reviewer instead of skipping it. Repeat until your own self-review pass is clean, then push. (gha#219/#220: one review round surfaced five findings — a DRY duplication, an incomplete-coverage doc overclaim, a wrong changelog category, an uncited claim, and missing test coverage for new logic — all catchable this way, since each was a direct match against gha's own `CLAUDE.md` conventions, not new information the review surfaced.)

Pre-push checklist

Pause point: after committing, before `git push`. Do-Confirm — confirm all seven here, in whatever order suits the round, with one exception: the items that **edit** the diff have to precede the items that **measure** it. Regenerating a generated tree and merging `main` change which lines are added, so an added-lines scan or a deleted-lines read taken before either is an answer about a diff you no longer have. The same holds *inside* item 3, which bundles two checks: reflowing a long line to clear its multi-sentence half retires the very lines its punctuation half scanned, so satisfying one check expires the other's result (`semantic-line-breaks`). Per `skill-checklists`; every item below exists because the bullets in this fragment record it failing at this exact boundary.

- ☐ **The whole test suite ran**, not the files you predicted the change touches, and the tests/failed/**skipped** triple was read — a non-trivial skip count means re-running with the gating flags set (`NOT_CRAN=true`, and whatever else un-gates a conditional skip).
- ☐ **Generated trees were regenerated** if the diff (or a `main` merge) touched a generator's inputs, and the PR body states how many changed files are generated.
- ☐ **Added lines were scanned** for banned punctuation and multi-sentence lines, run *after* committing, *after* every pass that edited the diff (your own reflow included), and with the three-dot range (`origin/main...HEAD`) — a pre-commit run reports on the wrong tree, a later edit retires the lines an earlier run scanned, and a two-dot range re-attributes whatever `main` deleted to you.
- ☐ **The changelog entry and the PR description were re-read** against the new behavior, not just the code — neither is in the diff, so no reviewer and no grep will catch a stale one.
- ☐ **The diff's deleted lines were read** (`git diff origin/main...HEAD | grep '^-'`), and each one was a decision rather than collateral from an edit's blast radius — a reviewer reads every deletion as deliberate and will rationalize an accidental one.
- ☐ **`main` was merged in** if it moved, with version parity re-checked afterward, so the round costs one review run rather than two — and any whole-file count a merge can worsen (spliced changelog bullets) compared before against after, since a defect caused by a *deleted* line is invisible to every added-lines check (`sync-with-main`).
- ☐ **Killer item: the push landed.** `git rev-parse HEAD origin/<branch>` agree before any reply asserting a fix. This one is marked because its failure is not an omission but a **false claim about state**, which a reviewer has no reason to doubt: CI reports green because it correctly validated the older head, and the session's own recollection agrees with the reply.

A clean verdict does not certify that your diff contains only what you meant, because a reviewer cannot tell an accident from a decision. Every check above tests whether the diff is *correct*. None of them tests whether it is what you *intended*, and those come apart whenever an edit does something extra — a replacement string that drops neighbouring

lines, a global substitution that rewrites more than the flagged occurrence, a stray hunk carried in from another branch.

A reviewer reads the diff as a set of deliberate choices, since that is the only thing a diff can present. So it does not report the extra change; it *explains* it, and often well — constructing a plausible rationale, grading the result appropriate, and moving on. That is worse than silence. Silence leaves the change unexamined, while a reasoned endorsement converts it into a decision the thread now records as settled, and any later reader finds an accident with an argument attached.

The tell is reading a review that justifies something you have no memory of choosing. Treat that as a prompt to check the diff rather than as confirmation, and note that the review's argument may be perfectly sound — the question is not whether the change is defensible but whether anyone decided it.

Deletions are where this concentrates, because an addition is something you wrote and a deletion is usually something that got displaced. `git diff origin/main...HEAD | grep '^-'` lists them in one command, and on a prose diff the list is normally short enough to read in full.

- **Do:** read your diff's deleted lines before pushing, and confirm each one was a decision rather than a casualty of an edit's blast radius.
- **Do:** say plainly, in the thread, when a review has blessed something unintended — the reviewer cannot know, and its verdict will otherwise stand as the record.
- **Don't:** treat a clean verdict as evidence about intent; it is evidence about correctness only.
- **Don't:** keep an unintended change because the reasoning offered for it turned out to be good.

When the edit is a regex or string patch rather than the Edit tool, two mechanisms turn that displacement into a silent over-deletion, and a self-check can wave both through. A non-greedy `.*?` DOTALL span binds to the **first** occurrence of its start anchor, so when that anchor is not unique the match runs from the wrong site and swallows every intervening structure up to the target suffix. And a self-check that counts only the **changed** element passes by coincidental balance while its neighbours are gone: deleting one sibling append and adding one target append leaves the target count unmoved, so the count reports success over a diff that dropped whole intervening blocks. The assertion that actually catches it verifies that neighbouring structures **survive** — the sibling loop still present, untouched element counts unchanged — rather than that the changed element's count is as expected.

- **Do:** anchor a string or regex patch on text unique to the intended site, and prefer the Edit tool's exact-string matching over a broad `.*?` DOTALL span.
- **Do:** make a patch self-check assert that neighbouring structures survive — the sibling function or loop still present, untouched element counts unchanged — not merely that the changed element's count is as expected.

- **Don't:** trust a `re.sub(..., flags=DOTALL, count=1)` whose non-greedy `.*?` start anchor is non-unique; it binds to the first occurrence.
- **Don't:** read "the assertions passed" as "the patch is correct"; a count-based check can pass by coincidental balance, and the `git diff` deletion-review is the real gate.

A clean verdict does not discharge the self-review against project conventions either, and the reviewer's own "not a finding" is where that shows up. The section above covers an **accident** a reviewer explains rather than reports, and its check is reading the diff's deleted lines. Here that check finds nothing. The choice is deliberate, nothing was displaced, and the diff contains exactly what you meant — it simply violates a rule stated verbatim in the repo's own `CLAUDE.md`.

A reviewer can notice such a choice, analyse it correctly, and still grade it acceptable, because it is judging whether the code is defensible rather than checking it against the project's written rules. That verdict arrives under a heading like "Observations (non-blocking)" and closes "Not a finding", which is stronger than the severity labels `address-every-comment` already warns about: "nit" downgrades an item, while "not a finding" retires it. So the part of a review most likely to be skimmed is the part a genuine convention violation is most likely to sit in.

The pre-push self-review this fragment already requires is the only thing that catches it, and a clean external verdict is exactly what makes that step feel finished.

- **Do:** re-run the project-conventions check against your own diff after a clean verdict, not only before the push.
- **Do:** read a reviewer's "observations" and "not a finding" items as candidate violations, and grep `CLAUDE.md` for whatever they discuss.
- **Don't:** let a reasoned "belt-and-suspenders is fine" settle a question the repo already answered in writing.
- **Don't:** treat a non-blocking label as deciding whether an item gets checked at all.

Proactively self-correct a technical claim you already told a reviewer, the moment further testing shows it was wrong — don't wait for the reviewer to catch it. If you stated a rationale (an approach is safe, a risk doesn't apply, a backstop exists) and then discover through your own follow-up verification that it's false, post the correction with the actual evidence immediately, rather than leaving the stale claim standing until a review round re-raises it. This keeps the review loop converging instead of churning on a claim you already know is wrong. (d-morrison/rme#989 / ucdavis/epi204#363: after telling both reviewers `references.bib` didn't share `CLAUDE.md`'s union-merge corruption risk, a follow-up merge simulation showed it does — posted the correction with repro steps on both PRs before either reviewer re-raised it.)

A fix is not "pushed" until it is on the PR's head commit — verify with a SHA comparison before telling a reviewer you pushed it. From inside a session, an edited working tree and a pushed commit feel identical, so a round that edits the files, writes the reply, and never runs `git push` produces a reply asserting a fix that does not exist on the branch.

Nothing contradicts it: CI reports green, because it correctly validated the older head; the next review round reviews code without the fix; and the session's own recollection of having made the change agrees with the reply. That makes it worse than an ordinary wrong claim — it is a false statement about *state*, which a reviewer has no reason to doubt and no cheap way to check. Before posting any reply that asserts a push, compare `git rev-parse HEAD` against the PR's own `head.sha` (`pull_request_read get`); if they differ, push first, then reply naming the real SHA. Run the same comparison in every periodic check-in on a PR you are babysitting, since the failure is silent and survives each round until something explicitly looks for it. This is the `algorithmatize-checks` rule applied to your own claims: two SHAs decide it exactly, so never substitute recollection.

A SHA you put in a PR body or a reply must be read, never recalled — and the PR body is where an invented one survives longest. The bullet above asks whether the right commit reached the branch. This asks the prior question: whether the commit you named exists at all. A short SHA is seven plausible hex characters, so writing one from memory feels like recalling a fact rather than asserting one, and the result is indistinguishable from a correct citation — nothing renders differently, and GitHub neither linkifies nor validates a SHA with no commit behind it.

The PR body is the worst host for it, for the reason `address-every-comment` gives about stale paraphrases there: the body is in no diff, so no reviewer reads it as part of the change and no `grep` over the diff finds it. It is also what a maintainer reads while deciding whether to merge, so an invented SHA misdirects the one reader most likely to act on it.

One command against the value you are about to paste settles it:

```
git rev-parse --verify <sha>^{commit}   # or: git cat-file -e <sha>
```

When a wrong SHA has already been published, correct it **visibly** rather than overwriting it silently — a reader who saw the original cannot otherwise tell a revised body from one that always said this, which is the same reasoning the withdraw-a-stale-blocker bullet below applies to a retracted caveat.

This is the commit-SHA case of the rule `report-mistakes-proactively` states for issue numbers ("never name an issue number before the issue exists"). Same defect, different artifact: an identifier guessable enough to assert casually, with nothing in the repository to contradict it.

- **Do:** read every SHA you cite out of `git rev-parse` or `git log`, and confirm it resolves before pasting it.
- **Do:** correct a published wrong SHA with a visible note naming the real one.
- **Don't:** write a short SHA from recollection because it looks like the commit you just made.
- **Don't:** expect review to catch it — a reviewer has no reason to suspect a citation, and the body is not in the diff they are reading.

The read side of that comparison can lag a push by a few seconds, so test the two *local* refs against each other before concluding anything failed. The rule above is an `algorithmatize-checks` case because two SHAs decide it exactly. That holds only as far as both numbers are current, and one of them is fetched over the network: `gh pr view <N> --json headRefOid` can still report the **previous** commit immediately after a successful push.

The failure direction is a false alarm rather than a false all-clear, so it is the safe one — but the reflexive response to it is wrong twice over. The rule's own remedy is "if they differ, push first", which is a no-op here, and treating a healthy branch as broken invites an amend or a force-push that manufactures the problem the check was watching for. A check that cries wolf on a clean branch also stops being run, which is the objection `algorithmatize-checks` raises against any instrument whose threshold cannot be trusted.

Local refs cannot lag this way, so they settle it:

```
git rev-parse HEAD origin/<branch>   # both local reads
```

- Equal — the push landed, and any disagreement with the PR API is a read-side artifact. Re-read it, preferably on a different surface, rather than re-pushing.
- Different — a genuinely unpushed commit, which is the case the rule exists for.

Note also that `git push` answering `Everything up-to-date` is itself evidence the remote branch already carries the commit.

- **Do:** compare `HEAD` against `origin/<branch>` first when the PR API disagrees, and re-read rather than re-push when those two agree.
- **Don't:** amend, force-push, or re-commit on the strength of an API SHA alone.

A brand-new branch can read back at the wrong commit, so the local two-ref comparison above is not sufficient there. That bullet offers `git rev-parse HEAD origin/<branch>` as the pair that settles the question, on the grounds that local refs cannot lag. They cannot. What they can do is agree with a remote ref that reads back at the wrong commit, and then the pair reports the push as landed.

The failure direction inverts, which is what makes this worth separating. A read-side lag is a false alarm, correctly called the safe one there. This is a false all-clear, arriving on the one instrument that rule offers for deciding the question.

The gap is in the trigger rather than in the remedy. The original SHA comparison does catch this whenever it is run. Nobody runs it after a `git push -u` that printed `* [new branch]`, set the upstream, and exited 0, because that is the least suspicious moment in the round.

`git ls-remote origin <branch>` reads the remote directly rather than through a tracking ref, so it is the instrument that catches it. The corrective push is an explicit refspec: `git push origin HEAD:refs/heads/<branch>`.

The likeliest explanation is a local one, and it reproduces offline. `git push -u origin <branch>` pushes the **branch ref**, not `HEAD`. So a local branch left behind at `main`'s tip, while `HEAD` carries the new commit, produces this whole signature with nothing on the server going wrong. Reproduced in a local bare repo, where no replica and no race exists: the push printed `* [new branch]` and exited 0, `git ls-remote` and the tracking ref both read `main`'s tip, `main..<branch>` held zero commits, and `git push origin HEAD:refs/heads/<branch>` then reported a real range. That last command is a test as much as a fix, since it answers `Everything up-to-date` when the branch ref and `HEAD` already agree.

Two things weigh against the read-side story, which an earlier draft of this entry weighted equally against the write-side one. A lagging replica cannot invent a value for a ref that never existed before, so its failure mode is the ref reading **absent** rather than reading one specific wrong commit. And a tracking ref is set from what the push sent, which makes its value a client-side fact rather than a later network read.

What stays genuinely unsettled is narrower than either reading claimed: the branch ref's own value at push time was never recorded, so the local explanation is the best supported one rather than a proven one. Note the shape of that, since it is the failure this entry is about. The entry has now over-claimed twice, first asserting a write-side fault, then asserting a parity between two hypotheses that the record does not support either. The practical advice survives all three readings, because the checks below are cheap whichever is right.

Two things about diagnosing one of these. The downstream error misdirects, because opening the PR fails with `No commits between main and <branch>`, which names a base-versus-head relationship and sends you to check the wrong argument. And `* [new branch]` here is the ordinary output for a branch that did not exist before, **not** the deleted-underneath-you signal `CLAUDE.md`'s "Use the existing PR branch" section describes. There the line is diagnostic precisely because the branch had already been pushed to; here it is expected, so the two cases must not be conflated.

The wrong value is the informative part, and it reads at first like noise. A race or a server fault has no reason to land on `main`'s tip in particular, whereas a branch ref cut from `main` and never advanced sits there by construction. So read a wrong value that happens to equal `main`'s tip as pointing at the local ref rather than at the network.

- **Do:** run `git ls-remote origin <branch>` after the first push to a new branch, and compare its SHA against `git rev-parse HEAD`.
- **Do:** run `git rev-parse HEAD <branch>` first when those two disagree, since a branch ref left behind accounts for the whole signature (and note that `--short` rejects a second revision, so pass neither).

- **Do:** re-run plain `git ls-remote` as well, so a ref that self-corrects stays distinguishable from one a re-push repaired.
- **Do:** re-push with `git push origin HEAD:refs/heads/<branch>` when the mismatch persists, and read the SHA range it prints as the confirmation.
- **Don't:** treat a `git push` that exited 0 and printed `* [new branch]` as evidence the commit reached the remote.
- **Don't:** assume `git push -u origin <branch>` sent the commit you just made – it sends the branch ref, which `HEAD` may have moved past.
- **Don't:** credit a corrective re-push with having repaired a remote-side fault when neither of those two controls was run.
- **Don't:** answer a `No commits between main and <branch>` error by re-checking the base branch argument before checking where the head ref actually points.

The same false claim arrives as *incoming* state when you pick a PR up mid-flight, and there the SHA comparison usually has nothing to compare. The bullet above governs a claim you are about to make. Its mirror is the claim already sitting on the PR when you arrive: the latest comment says which findings are fixed, so starting from it means starting from a summary rather than from the branch. The rule is the same either way, and only the *instrument* differs, because a claim written for a human rarely carries a SHA to check. "Three fixes landed in two commits" names files, not commits.

So compare the claim against the file list, which decides it in one call:

```
gh pr diff <N> --name-only   # DIFF_PR
```

A named file absent from that list was not touched, whatever the comment says. Nothing else in the PR contradicts a claim like this, and the trap is that green CI reads as corroboration when it is nothing of the sort. The checks that would have exercised the claimed fixes are frequently the very checks those fixes were supposed to add, so their absence is the reason CI is green.

The right response is to do the work rather than to dwell on the discrepancy. Say once, in the round's summary, that the earlier claim was not true of the branch, since a reader who saw it will otherwise assume the round was redundant.

- **Do:** run `gh pr diff <N> --name-only` against any inherited "already fixed" claim before deciding a finding is closed.
- **Do:** state plainly in your own summary that the prior claim did not hold, and name the head it was false at.
- **Don't:** treat green CI as evidence that a claimed fix landed.
- **Don't:** infer that a finding is stale because a comment says it was addressed.

Run that same command before *any* readiness claim, not only against an inherited one — a PR whose branch carries no implementation is green on every check. The

bullet above uses `gh pr diff <N> --name-only` **differentially**: it has a claim naming files, and it asks whether those files are in the list. That test needs a claim as input, so when nobody claimed anything it never runs, and the readiness path is exactly the case where nobody has. The **existential** question — does the list have anything in it at all — is the one no rule was asking.

`pr-on-claim` manufactures the hazard by design, and is right to: it opens the PR from `git commit --allow-empty` so the branch has a diff before any code exists. Merge `main` in later and the branch carries two commits, a real history, and no implementation. Every instrument then works perfectly and certifies nothing, because a check that finds no fault in an empty diff and a reviewer that raises no finding against one are both answering a narrower question than the one being asked.

Note which nearby checks *pass* on such a branch, since their passing is what makes the state feel verified. `git rev-parse HEAD origin/<branch>` agree, so the pre-push checklist's killer item is satisfied. The branch is not behind `main`. `get_commits` returns two, so a sweep keyed on zero commits does not flag it. `fully-clean`'s two criteria are each satisfied **maximally** by an empty diff, since neither has a term about content.

- **Do:** run `gh pr diff <N> --name-only` before reporting a PR ready, and read the returned paths against what the PR says it does.
- **Do:** treat an empty return, or a return holding only a `main` merge's incidental paths, as the PR carrying no implementation.
- **Don't:** count the claim commit or a `main` merge as work — neither is implementation, and both give the branch a plausible history.
- **Don't:** read all-green CI plus a finding-free review as evidence a PR contains anything; on an empty diff that is the expected result.

When the change affects downstream consumers, validate it against a real consumer repo before reporting the PR ready — a package's own test fixtures are built to exercise its code, not to resemble the packages that will actually use it. Fixtures are minimal by construction and tend to share one shape, so whole branches of new code can be structurally unreachable from them. A real consumer brings the input variety fixtures lack, and it is usually one clone plus one command to check.

This bullet, and the two further down that also turn on fixtures, are all about **coverage** — a fixture too thin to reach the code. `fixtures-are-not-evidence` covers the opposite direction: a fixture that works perfectly, and an inference drawn from its behaviour back to the real system it stands in for.

Three classes of gap this catches, none of them findable in a fixture:

- **Input shapes no fixture happens to contain.** A real package carries metadata the fixtures never needed — an entry of a different kind, an extra tag, an unusual name — so a branch written for it has never actually run on real input.

- **Message formatting under real counts.** Fixtures usually trip the plural path; a real repo hitting the same code with exactly one item exercises the singular wording, which no test asserted.
- **The migration/upgrade path, as opposed to the fresh-install path.** This is the one fixtures can never reach: a fixture is created new by the test, so it always gets the current templates. An existing consumer has the *old* config, and whether the feature reaches it at all is a different question from whether it works. Verify the claim in the changelog by running the documented migration step, rather than describing it.

Do it against a throwaway copy and push nothing to the consumer; the deliverable is evidence in the PR, not a change there. Record what the run covered in a PR comment, so a reviewer can see which paths real input reached. (d-morrison/altdoc#34: running the new reference-index generator against `d-morrison/rpt` covered a `\docType{package}` topic, the singular form of a missing-topic warning, and the documented "existing settings files do not pick this up automatically" caveat — confirmed by the page generating while `grep -c reference.html docs/index.html` returned `0`. None of the three were reachable from the repo's own fixture packages.)

Verify a blocker you assert in a PR body or a reply, with the same rigor you apply to a reviewer's claims — a stated blocker becomes a premise other people build on. The reviewer-facing checks above all point outward: verify the suggestion, verify the literal, verify the push landed. The inward case is easier to miss, because a limit you hit yourself feels like an observation rather than a claim. It is still a claim, and writing it into a PR body publishes it as settled fact: a reviewer reading "the pinned tool is unavailable in this sandbox" will reason from it, recommend a follow-up around it, and never re-test it, so one unverified sentence quietly redirects the review. Before asserting that something is unavailable, blocked, or impossible here, actually attempt it once — an install, a fetch, a single command — and say what you tried. A negative result from one incidental symptom (a failed version query, a single 403) is evidence the thing is not *already set up*, not evidence it cannot be. When a blocker you published turns out to be false, correct it where it was published, not only in the thread that surfaced it.

Attempting the base form of a command is not attempting its variants — a refusal describes the invocation you ran, never the flag you did not try. The rule above is discharged by one attempt, which is what makes this the harder miss: the attempt genuinely happened, so the instinct that rule exists to trigger has already fired and reported itself satisfied. What ships anyway is a claim about a *different* command — the same one plus a flag — for which no attempt exists at all.

The error message is what makes that generalization feel safe, because it is usually phrased about the operation rather than about the invocation. A refusal reading `cannot be moved or removed` sounds like a statement about the whole family, and it can even be half-true: that wording really is unconditional for one of the two operations it names. A half-true message is worse than a plainly wrong one, since re-reading it confirms the reading you already had.

So before writing that something is impossible, check whether the tool's own `--help` or documentation offers a flag for exactly this case, and run that form too. One `--force` is cheaper than the correction round, and it is the only thing that turns "the command refused" into "the operation cannot be done".

- **Do:** run the flag variant the docs or the error itself name, before generalizing a refusal into an impossibility.
- **Do:** scope the published claim to the invocation actually run, naming the exact command and what it exited with.
- **Don't:** read an unconditional-sounding error as covering flags you never passed.
- **Don't:** count an attempt at the base form as discharging the rule above for a variant of it.

Name the specific gate when you report a blocker, not a category word that happens to be one of several. The verify-a-blocker rule two sections above governs *whether* something is blocked, and its remedy is to attempt the thing once. This governs *why*, and it fires after that remedy has already succeeded: the call was attempted, it genuinely failed, and the blocker is real. Only the attribution is wrong, which is why nothing about it feels like an unverified claim – the part that usually goes unchecked has, this time, been checked.

The hazard is a platform with two gates whose refusals read alike. `resolve_review_thread` on a transferred repo fails under either spelling of the owner, saying `Access denied` both times, for unrelated reasons: the old owner trips a comparison between the thread's node and the declared `owner/repo` string, and the new owner trips the session's own repository allowlist. Only the second of those is scope. So "blocked for scope reasons" is not a loose summary of the first. It names a mechanism that was not involved, and it names one that genuinely exists on that platform, which is what lets it survive re-reading.

That last point is the whole cost. A category word that is also the proper name of one mechanism cannot double as the generic term for its family, because a reader cannot tell which you meant, and the wrong reading is actionable: someone told a call failed on scope will reach for the other owner, which fails too. Quote the error's distinguishing clause instead of classifying it. The quote is usually shorter than the paraphrase, it is checkable, and it stays correct even when your model of the platform is not.

- **Do:** quote the clause that distinguishes the failure, and name the gate it belongs to.
- **Do:** re-read a blocker you have restated several times, since a paraphrase repeated across status reports hardens into the record.
- **Don't:** use one mechanism's own name as a generic word for its category.
- **Don't:** treat having verified *that* something is blocked as having verified *why*.

When the blocker is a hang, inspect the process rather than re-guessing what it is waiting on. The bullet above governs a real failure whose gate was misnamed from an error message, so there is at least a message to re-read. A hang gives you nothing to quote and nothing to classify, and that vacuum gets filled by a guess about mechanism. The guess then

arrives feeling like a measurement, because something really was run and something really did block.

"It needs a TTY" and "it hangs when run non-interactively" are both category words in the sense the bullet above means. Each names a plausible mechanism, neither was observed, and a reader cannot tell which one you checked. Replacing the first with the second after a probe returns nothing feels like progress, and moves you no closer to a gate you can name.

A blocked process answers the question directly, and the reads are cheap:

```
ps -o pid=,stat= -p <pid>        # S: alive and blocked, rather than spinning or gone
lsof -p <pid> -a -d 0            # what fd 0 actually is: tty, pipe, or socket
ps -o ppid=,command= -p <pid>    # what launched it, and with what arguments
```

Those turn "it hangs" into a specific, checkable fact about *what* it is waiting on, which is the gate the bullet above asks you to name. They also separate two states that no amount of re-running distinguishes: a capability check refusing at startup, and a read reached only late in the flow after the interactive step succeeded. Those call for opposite responses, so guessing between them is not a harmless imprecision.

- **Do:** read `ps -o stat=`, `lsof -d 0`, and the process tree before describing what a hung command is waiting for.
- **Do:** say which read produced the answer, so the gate is checkable rather than asserted.
- **Don't:** substitute one guessed mechanism for another because a probe produced no output.
- **Don't:** report a timeout signal as evidence about *why* something blocked; it is evidence only that it had not finished.

Do not reach for the probe first, either. The probe that produces the hang is itself a live command with its own first instant, and running an interactive one to see what it does is the failure growth-mindset's "A timeout bounds how long you wait" section covers.

A blocker that was true when you published it can stop being true while the PR is open, and withdrawing it is your job, not the reviewer's. The verify-a-blocker bullet above covers a blocker that was never true, and the "Name the specific gate" bullet covers a real blocker whose mechanism was misnamed. This is the harder case, because the caveat was correct and diligent when written, so nothing about it reads as a defect later — and a sentence saying "this could not be checked" is one nobody re-checks, least of all the reviewer, who has no way to know the environment moved. It keeps steering the review regardless: a verdict can repeat the caveat back as an accepted limitation, which makes the stale claim look corroborated. So when the cause of a blocker changes — a host unblocked, a tool installed, a quota reset, a dependency published — re-run the check and withdraw the caveat where it was published, saying explicitly that it is withdrawn rather than quietly deleting the sentence. A

reader who saw the original needs to know it was retested, not be left wondering whether it was ever true.

A `main` merge is one moment that must fire this check, because it can falsify one of your own hedges without producing a conflict in the file that carries it. After merging `main`, run a whitespace-normalizing search over the PR's touched files for hedge forms such as `still open`, `not yet merged`, `once that merges`, `as of`, `will live at`, and `proposed in`. Do not use line-oriented literal grep in this semantic-line-break corpus: a phrase split across lines is exactly the case this check must still find. Re-check each hit against the new base before pushing the merge. The conflict marker is not the scope of the review: a cleanly merged file can be where the stale caveat lives.

- **Do:** after every `main` merge, scan the PR's touched files for merge-status hedges with whitespace-normalizing search, then re-read each hit against the new base.
- **Don't:** assume a hedge survived because the file that contained it merged without conflicts, or because literal grep missed a phrase split across semantic lines.

Landing a fix falsifies whatever prose documented the defect, and that prose is never in your diff — so grep for it rather than expecting to be reminded. The blocker-withdrawal rule above, illustrated by ai-config#774, covers a caveat **you** published on **this** PR, which the environment then moved out from under. This is the case where you moved it yourself, and where the stale text lives in the standing corpus rather than on the PR: a memory bullet describing the hazard, a README warning about it, a docstring asserting the behaviour you just changed.

The sync rules in `address-every-comment` all end at the PR's own artifacts — the changelog, the PR body, a skill's inline restatement — and each prescribes grepping the diff. Documentation of a defect cannot be found that way, because not being in the diff is the entire property that makes it survive. So the trigger has to be the fix itself, and the search has to leave the files you edited.

Two shapes, and the second is worse because it was never true.

- **Prose staled by the fix.** It was accurate when written, so nothing about it reads as a defect, and a workaround it prescribes becomes active misdirection the moment the thing it worked around is gone. Keep the entry where the old behaviour explains something — most of a corpus is written against it — but mark plainly that it is history and name the change that ended it.
- **Prose asserting conformance to a reference.** A docstring saying the code "follows" some reference implementation is a claim about two artifacts, and your own divergence falsifies it. This one is not staleness at all: it was false before you arrived, and it is load-bearing, because a reader checking the code against the reference stops at the sentence saying someone already did.

- **Do:** grep the repository for the defect, the workaround, and the behaviour you changed, before calling a fix complete.
- **Do:** mark a superseded entry as history and name the change that ended it, rather than deleting it, when the old behaviour still explains other text.
- **Don't:** treat a clean grep over the diff as coverage — the stale prose is outside it by construction.
- **Don't:** leave a doc asserting conformance to a reference standing when the code diverges; correct the claim in the same change that establishes the divergence.

An instruction's own suggested code is not exempt from the project-conventions self-review above. The self-review rule assumes you wrote the diff; a snippet handed to you in an issue, a task description, or a design doc slips past it, because adopting someone else's suggestion does not feel like authoring. It is authoring — once pushed, it is your diff, and the project's conventions bind it exactly as they bind anything you wrote yourself. Run the same convention check over borrowed code before pushing it, especially when the suggestion is a plausible-looking one-liner and the convention it breaks is documented rather than linted. (d-morrison/altdoc#73: the issue proposed ending a function with a bare trailing `hashes`, which reads as a fix for the fragility it names but is still an implicit return, so a statement added after it silently becomes the return value. The lab manual asks for an explicit `return()` regardless. Review caught it; the project's own stated convention would have, one step earlier.)

When the code path under test has a staging or transform step between input and output, a passing unit suite is not evidence it works — exercise the real path once. Fixtures instantiate the shape the test author had in mind, so a wrong assumption about *where* the code runs is invisible to every one of them: the tests and the bug share the assumption. This is the same gap the downstream-consumer rule above covers, one level in — there the missing variety is the consumer's input, here it is the pipeline's own directory layout, timing, or intermediate representation. One real invocation is usually cheap, and it tests the assumption the fixtures encode rather than re-confirming it. (d-morrison/altdoc#76: a guard checked for the copied logo under `docs/`, but the `quarto_website` path stages into `_quarto/` first, so the logo line was dropped on every render of the one generator the feature wired up. Seventeen unit assertions passed throughout; one throwaway render found it immediately.)

When new code branches on a third-party tool's behavior, read that tool's own config or docs for the specific behavior — don't infer it from what the tool broadly does. The bullet above covers your own pipeline's layout; this one covers the tools that pipeline drives. An inference of the form "it builds HTML, so link to `.html`" is exactly the shape that feels too obvious to check, and a tool's defaults routinely contradict it. Two properties make this worse than an ordinary wrong guess. The inference usually lands in a branch your own fixtures cannot reach — you have no fixture for someone else's renderer — so the test suite agrees with you. And it produces output that is well-formed and plausible (a link, a path, a flag), so a reviewer skimming the diff has nothing to catch, and the failure surfaces only in a

consumer's published site. Name the setting you are relying on, and check its actual default before writing the branch.

A regression test written alongside a fix can lock the bug in rather than catch it — assert the two paths that diverge, not the one you just touched. A test authored in the same pass as the code tends to record what the code *does*, because you run it, see it pass, and move on. That is usually harmless. It becomes a lock when the fixture is thin enough that the buggy and the correct path produce the *same* output: the assertion then encodes the degraded result as intent, and every later reviewer reads a green suite as evidence the behavior was chosen. The next round's finding lands on your test, not just your code.

The tell is the same each time: **a fixture missing the input variety that makes the two paths differ.** So when a bug is an asymmetry — nested versus top-level, second render versus first, one generator versus another — build the fixture so both sides are present and assert them together. Either side alone is unfalsifiable, since the case that reveals the bug is the *comparison*. Then prove it: revert the fix and confirm the new test actually fails. A regression test never seen to fail is a guess about what it covers.

A systematic audit done by skimming is worse than the one-at-a-time version it replaces. Batching a check — "rather than wait for the next round to find divergence number four, compare all four at once" — is the right instinct, and it inverts if each lookup gets less care than it would have alone. Two things make the batched form more dangerous, not less. Its output is usually a claim recorded somewhere durable (a comment, a doc, a table), so an error is published rather than merely held; and it arrives labelled *audited*, which is precisely the word that stops the next reader from checking. A wrong comment in a block written to prevent a specific future change invites that change while appearing to forbid it. Concretely: when the thing being audited is a function, grep for the function, not for a pattern in its file — a file with several functions will hand you the first match, which is often not the one you mean. Name the function in whatever you write down, so the claim stays checkable.

Adding an explanation supersedes whatever the file already said about the same thing, so re-read the older passage — your own diff is the likeliest source of a contradiction nobody flags. The sync rules in `address-every-comment` all fire on an external trigger: a reviewer quotes a phrase, or behavior changes and a changelog goes stale. This one has no trigger at all. You add a paragraph explaining that something was misunderstood, and the note recording the original misunderstanding sits a few lines below, still stating it as fact. Nothing conflicts, no check fires, and both passages read plausibly on their own — but a reader who reaches the older one first comes away with exactly the belief the new text was written to remove.

The tell is a diff that adds an explanation, a correction, or a "what this actually means" paragraph near existing prose. Re-read the surrounding passage as a whole rather than diffing your addition in isolation, and treat a historical record ("we observed N of X") as a claim your explanation may have just falsified. When the older passage recorded a *different* session's

observation, correct it with reasoning that stands on its own rather than restating it as though you had seen it — an inference presented as an observation is the same defect one level up.

The same rule applies within a single diff, and there nothing prompts the check at all. The version above compares your addition against the *existing* file, so re-reading the surrounding passage catches it. The harder case is a diff that both adds an explanation arguing against some older wording **and** rewrites that wording, in the same changeset. Then the argument survives while its target does not, and every file still reads plausibly on its own: the new prose is coherent, the rewritten passage is coherent, and only the cross-reference between them is stale. There is no older text to go back and re-read, which is the cue the other version relies on.

So when a diff rewrites a passage, grep the rest of the diff for references to what that passage used to say, not just for references to the file. A rebuttal of the form "the section below already says X" is the shape to watch, since it pins the argument to wording the same commit may be deleting. Prefer stating the anti-pattern directly over citing another section as the thing being argued against; a self-contained sentence cannot go stale when its neighbour changes.

And when the explanation you add is a *mechanism* claim, test the class it distinguishes, not just the sample in front of you. The bullet above is about contradicting old text; this is about the new text being unfalsifiable on the evidence you gathered. A classifier validated on a population containing no positive instance of the class it is supposed to catch will report a clean result either way, so "it returned zero" is not evidence it works — it is the same missing-input-variety tell the regression-test bullet above describes, moved from a fixture to a diagnostic. Ask what a true positive would look like, confirm one exists in what you tested, and if none does, say so instead of claiming the mechanism separates the cases. (ai-config#770, same day: a `git log -- skills/<name>` probe was said to separate "deleted from the repo" from "never ours" *exactly*, on the evidence that it reported zero false orphans. The repo contained no deleted-but-still-installed skill at all, so there was nothing for it to get wrong; and `git rev-parse --is-shallow-repository` returned `true`, meaning anything deleted before the shallow boundary would have been silently misread as harness-provided. The claim went into a PR reply before either check was run, and ai-config#765 had independently reached the correct conclusion.)

A symptom that stops reproducing is a fix having landed, until you have checked otherwise — reaching for nondeterminism is the attractive wrong answer. The bullet above governs a mechanism claim you write into a file. The same defect arrives in a status report, and there it is easier to publish, because "the check is just flaky" sounds like a complete explanation while resting on nothing. It is also unfalsifiable from a single observation and predicts nothing, which is exactly why it feels safe to say.

The shape to watch for: a known-failing check, a tracked false positive, or a reproducible bug goes quiet, and you explain the silence with a property of the *tool* rather than a change in the *world*.

Check for the merge first, because the instrument is a timestamp comparison and it costs one API call. Compare when a candidate fix merged against when each observation was made. That turns "it seems flaky" into a before/after table with a negative control — a far stronger claim than the one you were about to make, and one a reader can act on.

- **Do:** look for a merged fix, and date it, before attributing a vanished symptom to anything.
- **Do:** report the before/after with its timestamps, so the negative control is visible rather than asserted.
- **Don't:** explain a symptom's disappearance as nondeterminism on the strength of one clean run.
- **Don't:** carry such a claim into an issue or a decision doc, where it argues against the very fix that produced the silence.

Verify a command, path, or flag *you* write into a doc, with the same rigor `address-every-comment` demands for one a reviewer suggests. That rule and the verify-a-blocker rule above both point outward, at a claim someone else made or at a limit you hit. This is the one you author from scratch, and it is easier to miss than either, because inventing a plausible command does not feel like making a claim at all — it feels like remembering one.

The shape is a CLI invocation, a file path, or a flag written into documentation, a comment, or a registry, where the surrounding prose is carefully sourced and the literal is not. A reader reaching for it gets `unknown command`, and they get it while following a document whose every other sentence checked out, so they are likelier to doubt their own setup than the doc.

Settle it against the tool's own source or `--help` rather than recollection. For a CLI, the subcommand registration list is definitive and usually one fetch away, and it beats a docs page because it cannot lag the release you are describing. Quote what you checked in the commit message, so the next reader inherits the evidence instead of the assertion.

- **Do:** confirm every literal you invent against the tool's own source or help output before it lands in a doc.
- **Do:** cite the file or command you checked, so the claim stays falsifiable.
- **Don't:** infer a subcommand from a family that has its siblings (`gh label list`/`create`/`edit` does not imply `gh label view`).
- **Don't:** treat a literal as exempt because the prose around it is well-sourced — the literal is the part a reader executes.

Run that check over your own fix, too — the remedy for an unverified literal is where the next unverified literal goes. The rule above fires when you notice you are writing a literal. Answering a finding does not feel like that: it feels like careful work, and the care is real, so the fix inherits an assumption of rigor from the diligence of writing it. The specific rule you are in the middle of applying is therefore the one least likely to be applied to its own application.

A correction also tends to *add* literals rather than merely repair one. Citing a source properly means naming the tool, the flag, the version, the file — each an assertion, each as guessable as the one under review, and none of them the thing the finding was about. So the fix can carry more unverified surface than the original did.

Nothing external catches this. The reviewer sees a fix that addresses the finding and confirms it, per the clean-verdict entry above, and the thread then records the item as settled twice over.

- **Do:** re-run the rule you are applying against the text of your own fix, before committing it.
- **Do:** say in the thread when a fix's own draft tripped the same rule, since that is the only place the near-miss is visible.
- **Don't:** treat the effort of writing a correction as evidence the correction is verified.

The same rule reaches past a literal, to the defect CLASS a code fix just closed. The block above governs a correction to prose, where the artifact is a citation and the risk is one more guessable literal. A fix to code carries the same exposure at a larger unit: the change that closes an instance of a defect class is itself new code, so it can instantiate that class again, one layer down, in the very edit that removes it.

Nothing about writing the fix surfaces this. The finding names a site, the fix closes that site, and the diff reads as a strict improvement, so the question "does what I just wrote have the property I just removed" is never posed. The next round then reports the original class at a new address, and it reads as a fresh gap rather than as the previous fix's own residue.

A consolidation commit is the highest-risk host for it, and the likeliest to be trusted. Merging several drifted copies of a concept into one shared definition is a textbook DRY repair, and it makes the commit *feel* like the opposite of forking. That feeling is what carries a newly hand-rolled helper past your own review: a second concept can be duplicated in the same edit, and the de-duplication of the first supplies the whole commit's framing. Consolidating one duplicated concept is no protection against forking a different one beside it.

So after fixing an instance of a class, ask what the fix **added**, not only what it removed, and check the addition against the class. For a new helper the mechanical form is cheap: whatever shared definition the neighbouring code already composes for the same job, compose that instead of writing a fresh equivalent, so the new site inherits every property the old sites have and whatever they gain next.

- **Do:** ask whether the fix's own new code instantiates the class it closed, before committing it.
- **Do:** treat a commit that consolidates one duplicated concept as owing a check that it forked none, since its framing argues the other way.

- **Do:** compose an existing shared anchor or helper into a new site rather than hand-rolling an equivalent, so the site inherits later fixes too.
- **Don't:** read a diff that removes a duplicate as evidence that it added none.
- **Don't:** treat the next round's finding at a new address as a fresh gap without first checking whether your own previous fix created that address.

When regenerating a generated tree makes it most of the diff, say so in the PR body — otherwise a reviewer reads it as pollution and blocks. Two failures share one root here, and both cost a round.

The first is editing the generated file rather than its source. A generated file usually declares itself in a header, and that header is the last thing you read when you have already found the line you wanted to change; the generator then reverts the edit and the sync check fails. Grep for `generated by` before editing any file you did not create.

The second is what happens once you regenerate correctly. A one-line source change can rewrite hundreds of files, and a reviewer working from a truncated diff sees only the generated bulk — so the finding comes back as "revert the unintended changes", where complying would break the very check that demanded them. The PR body is the only place that can pre-empt this, since it is what a reviewer reads before the diff. Lead with the ratio and a per-path table marking each path generated or hand-written.

- **Do:** grep a file for a generated-by header before editing it, and change the source instead.
- **Do:** state in the PR body how many of the changed files are generated, and name the hand-written ones.
- **Don't:** revert generated output because a reviewer calls it noise — check first whether the sync check requires it.
- **Don't:** assume a reviewer sees the source files; on a large diff they frequently do not.

(Morrison-Lab/ai-config#834, same day: a fix was applied to the generated `tool-mappings.md`, which `sync-codex-skill-wrappers.py` then overwrote, failing `validate` with `stale tool-mappings.md`. Redoing it in `tool-mappings.yml` regenerated 175 `codex-skills/` wrappers, and Jules returned `VERDICT: block` twice for "bulk pollution", its second verdict noting it had read only a truncated diff. `claude-review` called the same finding a false positive at the same head.)

Run the whole test suite before pushing, not the files you predict the change touches — and check that the ones you ran were not silently skipped. The pre-push self-review above assumes your local green means something. Two things quietly hollow it out, and they compound: you choose a subset, and the subset then reports success without having run.

The subset is chosen by predicting blast radius, which is exactly the judgement the change calls into question. The tests that break are frequently *not* in the files you edited: a test elsewhere asserts the behaviour you are changing, often with a comment stating the rationale your diff

invalidates. Nothing about editing `R/foo.R` suggests opening `test-bar.R`, so the prediction feels complete while omitting the one file that matters.

The second is worse, because it looks like evidence. A conditional skip — `skip_on_cran()` without `NOT_CRAN=true`, `skip_if(!.venv_exists())`, `skip_if_offline()` — turns the test that would have caught the bug into a pass. `devtools::test()` sets `NOT_CRAN` for you — it applies `withr::local_envvar(r_env_vars())`, and devtools documents that set as "the standard environment variables set by devtools", singling out `NOT_CRAN` as "of particular note for package tests" (`R/test.R`, `R/check.R`). `testthat::test_file()` and `testthat::test_dir()` do not, so the harness you reach for by hand for a quick targeted run is exactly the one that skips. Setting it explicitly anyway (`NOT_CRAN=true Rscript -e '...'`) costs nothing and is what the rest of this corpus does. So read the **skip count**, not just the failure count: a run reporting `0 failed, 20 skipped` has told you almost nothing, and is indistinguishable at a glance from one that verified everything.

Match the environment the change will be judged in, too. Running with an env var set that CI does not set (or vice versa) reproduces a *different* configuration, and its failures and passes both mislead.

- **Do:** run the full suite before pushing, and state the tests/failed/ skipped triple rather than "tests pass".
- **Do:** set the flags that un-gate conditional skips, and re-run if the skip count is non-trivial.
- **Don't:** scope a local run to the files you edited — the test asserting the old behaviour is usually somewhere else.
- **Don't:** read a green subset as a green suite, or a skip as a pass.

Running a script is not running its tests, and an "advisory" check can have a hard-gating twin. The bullets above assume you reached for the test suite and took too little of it. The nearer miss is reaching for the **production script** instead: you touch something a checker measures, run that checker, read its exit code, and treat that as having verified the property. It feels like stronger evidence than a test, since it is the real instrument on the real data.

It answers a different question. A script **reports**; a job **gates**, and the gate is frequently a separate step asserting the same property about the repo itself. The two can disagree by design — one deliberately advisory, its twin deliberately blocking — so a script's exit 0 says nothing about whether the job is green.

What makes this worth its own entry is that the conclusion is easy to **publish**, and a claim about what CI enforces is the kind other people act on. Per `metacognitive-monitoring` that is a scope claim, and its remedy applies: check the population — every step of the job — rather than the sample that came to mind.

- **Do:** run every check the CI job runs, its test files included, before pushing.

- **Do:** grep the job definition for other steps touching the same property before saying anything about whether it gates.
- **Don't:** substitute a production script's exit code for its test file.
- **Don't:** infer a job's behaviour from one step's label — "(advisory)" describes that step, not the job.

A third failure mode of the whole-suite rule above: the suite holds no case that could have failed. The two hazards that rule names both assume a test aimed at the behaviour you changed exists — one you skipped by scoping the run, or one a conditional turned into a pass. Widening the run and un-gating every skip fixes those two and does nothing for this one, where the case simply is not there, because the defect class had not been conceived when the suite was written.

Provenance is the whole argument. A suite's case population was fixed before your change existed, so its green is **logically independent** of whether that change is correct. Red still carries information, since a suite that fails has found something. Green is not its mirror, and reading the two symmetrically is what turns a routine run into a verification claim.

That asymmetry is what makes such a report persuasive rather than obviously thin. Running the whole suite is real work and the diligent thing to do, and a line like `15/15 suites passed` is specific, checkable, and true. It is just an answer about the cases somebody wrote earlier.

So when the change is a **guard** — a matcher, a validator, a filter, anything whose job is to refuse a class of input — the verifying step is to construct that class yourself and run it against the pre-change and the post-change code, reporting the two behaviours side by side. Two columns over inputs you chose is a comparison; a suite total is not. This is `metacognitive-monitoring`'s "an instrument's answer is only as wide as its input" with a test suite as the instrument, and the construction step is `algorithmatize-checks`'s "never predict which case will fail; enumerate the class" applied to inputs rather than to a report.

Distinguish it from the neighbours it resembles, since all of them concern a test that was aimed at the question and fell short. `fixtures-are-not-evidence` governs a fixture that cannot discriminate; the regression-test rule earlier in this file governs a case you wrote in this pass and never saw fail; `dont-incur-technical-debt` governs a test that reimplements its own subject. Here nothing is defective. The suite is sound, and it was pointed somewhere else.

- **Do:** construct the input class the change is supposed to handle and diff its behaviour against the pre-change code, before calling a guard verified.
- **Do:** name which cases could have exercised the defect class, rather than quoting a suite total — the total is a fact about the suite, not the diff.
- **Don't:** offer a pre-existing suite's green as verification of a change it holds no case for; those cases predate the defect and cannot speak to it.
- **Don't:** read the tests/failed/skipped triple above as covering this — it makes the report more precise without making it any more relevant.

What "Fully Clean" Means

"Fully clean" is the terminal state the ARDI review loop drives toward. A PR/MR is **fully clean** when **both** of these hold (and verified via `python3 scripts/check-pr-fully-clean.py <pr-number>`):

Worked-example case records for the rules below live in `fully-clean.cases.md`, moved out of the auto-loaded context.

1. **All CI workflows and check runs are green AND completed.** Every workflow and check run passes — not just the required checks and not just the review job. "Green" means finished with a passing outcome (success or skipped), not merely "currently reporting green while still running" — never treat a workflow or check run that's still queued or in progress as clean, even if nothing has failed yet. **A reviewer's posted verdict does not mean the review check has finished, so don't let a clean verdict stand in for criterion 1 on its own job.** The bot posts its comment and then its run keeps going (bookkeeping steps, a cost tally, the gate job that consumes its result), so a full `Ready for merge` comment can sit on the PR for minutes while `claude-review` still reads `in_progress` and the `require-review` gate is still `queued`. Reading the verdict and moving straight to "clean" skips the very state this criterion exists to catch. The gap runs the other way from the stub-review case in `review-verdict-pitfalls.md`: there the check is green and the verdict is missing, here the verdict is real and the check is unfinished. Re-read the check runs after the verdict lands, not just before. (The exact field names and casing for these states differ by API surface — REST's check-runs endpoint returns lowercase `status`/`conclusion` strings like `completed`/`success`, while `gh pr checks`/GraphQL's rollup returns uppercase `state` values like `SUCCESS`; don't hard-code one casing when scripting a check.) **A raw Actions workflow run and a check run are not the same thing, and the usual lookups (`gh pr checks`, `get_check_runs`) only cover check runs (plus legacy commit statuses) — not every workflow run necessarily produces one.** A workflow run that's blocked on `action_required` (e.g. pending manual approval) before any job starts can complete with zero jobs and consequently zero check runs, making it invisible to a check-runs-only poll. This normally doesn't affect mergeability (GitHub's branch-protection required-checks gate operates on checks, not raw workflow runs, so a check-run-less run can't be wired as required), but if something about a PR's CI state looks off despite `gh pr checks` reporting all-clear, cross-check the raw workflow runs before trusting the checks-only view. **`gh run list --commit <head-sha>` is not a reliable substitute for this cross-check on its own**: it returns every attempt for that SHA (including superseded/cancelled re-runs, so an old failed attempt can look like an outstanding blocker), and a run triggered by `issue_comment` or a `workflow_dispatch` invoked without an explicit `ref` can be recorded against the default branch's SHA rather than the PR's head SHA and be missed by a `--commit` filter entirely. Neither `--commit` nor `--branch` is fully reliable for this, because GitHub itself does not record a reliable PR linkage for these trigger types:

an `issue_comment`-triggered run on this very PR (#635, run 29967418653) recorded `head_branch: main` and an empty `pull_requests` array via the raw REST API (`GET /repos/{owner}/{repo}/actions/runs/{id}`) — verified directly, not assumed — so no single filter (commit, branch, or the API's own PR-linkage field) reliably narrows these runs to the ones for this PR. Treat this cross-check as best-effort: `gh run list -R <repo> --workflow <name>` (unfiltered, or windowed by approximate timestamp) and eyeball for anomalies near when the PR activity happened, rather than trusting any one filtered command to be exhaustive. This includes non-gating checks like the Coverage / codecov job: don't merge around a red Coverage run just because it isn't a required check, unless there's a specific, stated reason for that merge (the project wants to maintain decent coverage, so a red Coverage job is a real signal to fix, not to ignore). **`codecov/patch` is a separate check from the repo's own Coverage workflow job, and both must be green.** The Coverage job runs the coverage-instrumented test suite; `codecov/patch` is the Codecov service's own status check, gating the PR's DIFF against a minimum patch-coverage percentage — a repo can have a fully green Coverage job while `codecov/patch` still fails (uncovered new lines in the diff). When delegating implement-a-PR work to a subagent, name this check explicitly in the brief ("ensure `codecov/patch` passes, not just the test suite") — a subagent that only runs the local test suite and checks it's green has no way to know it also needs to check a service-side status check unless told. **The set of checks is not fixed while the run proceeds, and a check run's name is not unique — so "the ones I was waiting on went green" is not the same statement as "every check is green".** A job can spawn further jobs when it completes, so the total grows *after* you started watching. Nothing announces that, and the natural mental model is a fixed list draining toward zero, which makes the growth invisible precisely when you are closest to declaring ready. Re-fetch the whole list each time and re-count it, rather than checking off the names you remember.

The name collision is the sharper half, because it turns a careless check into a confidently wrong one. Two check runs can carry the *same name* on the same head — an earlier one that already succeeded, and a later one still running — so matching on the name returns the stale green and reports the PR ready while the other is still going. They are usually not re-runs of each other: the common case is two separate workflow runs that each happen to define a job by that name, so neither replaces the other and both are legitimately present. Key on the check run's **id**, and read `status` before `conclusion`, since a run still `in_progress` has no `conclusion` to be misled by.

`status` itself can be stale, so never infer a job's *duration* from it. Reading `status` before `conclusion` is right, and it invites a second inference that is not: that a run still showing `in_progress` is still running, and therefore that the time since `started_at` is how long it has been going. The field lags. A job can read `in_progress` for minutes after it has actually finished, so "started at T, still in_progress now" measures the API's freshness rather than the job's runtime.

That is harmless while you are only waiting for a job to end, which is the usual reason

to read the field — the lag costs a poll. It inverts the answer whenever **duration is itself the diagnostic**. A reviewer job that dies on a bad credential and one that genuinely reviews a diff differ mainly in how long they take, so a stale `in_progress` is indistinguishable from exactly the recovery you are watching for, and it arrives as good news.

Take duration from the log's own timestamps — first line to `Cleaning up orphan processes` — or from `completed_at` minus `started_at` once the run really is complete. Both are facts about the job; `status` at any given moment is a fact about the API.

- **Do:** read elapsed time from log timestamps whenever the length of a run is the thing being judged.
- **Don't:** conclude a job is still running, or has passed some duration threshold, from `in_progress` plus the wall clock.

A `BlobNotFound` / HTTP 404 on the job-log fetch means the job has not completed, not that it has hung. The block above says to read a run's duration from its log timestamps. That remedy is unavailable while a job is still running, because there is no log to read yet: GitHub archives a job's log blob only when the job completes, so `gh api "repos/<owner>/<repo>/actions/jobs/<job-id>/logs"` (and the MCP `get_job_logs`) returns `BlobNotFound` / 404 until then. So a 404 there is evidence the job is still going, and reading it as a hang inverts the signal.

A still-in-flight job also legitimately reads `status: in_progress` with `conclusion: null`, and neither the 404 nor that status distinguishes a normal long-running review from a genuinely stalled one. Only completion settles it, or the live streaming log in the Actions UI, which is served before the blob is archived. So do not conclude "hung" or "produced no verdict" from a 404 plus an `in_progress` status; wait for the job to finish and read the verdict it then posts.

A bare 404 is ambiguous in one further way worth naming, because the two readings call for opposite responses. A job that *completed* with no logs at all — the ~1s concurrency self-collision in `debugging.md`'s "An Actions job that fails in ~1s with NO logs" section — also 404s on the log fetch. The discriminator is the job's own `status`/`conclusion`, never the 404: `in_progress` / `null` is still running, while `completed` / `failure` with `completed_at` stamped before `started_at` is that instant-fail case.

- **Do:** read a 404 / `BlobNotFound` on the job-log endpoint as "the job has not finished", and wait for completion (or read the live UI log) before judging its outcome.
- **Do:** take a job's real state from its `status`/`conclusion`, since the same 404 covers a still-running job and a completed-with-no-logs one.
- **Don't:** read a 404 on the log fetch as positive evidence of a hang or a stall — it is the opposite, evidence the job is still running.
- **Don't:** file an issue reporting a review job as hung or "no verdict produced" while its log fetch still 404s and its status is `in_progress`.

`gh pr checks` is not a complete enumeration of a head's check runs, so read the commit check-runs endpoint before deciding that everything has finished. This is a different gap from the workflow-run one above, and that gap's remedy is not the direct answer to it. The earlier paragraph warns that a workflow run may produce **no check run**, so a check-runs query cannot see it, and sends you to the raw workflow runs. Here the check run **exists** and the check-runs endpoint returns it. It is `gh pr checks` that omits it.

The raw-run route is not blind to this, but it is indirect, and every caveat that paragraph attaches to it applies unchanged. The omitted check does have a backing Actions run, so a `gh run list --commit <head-sha>` sweep can surface it under the run's own name rather than the check's — which means the raw-run sweep is a best-effort corroboration here, not the instrument to reach for. The check-runs endpoint names the check directly and answers in one call.

The failure direction is the expensive one, because the omitted check run can be `in_progress` while `gh pr checks` reports zero pending. Anything keyed on that count — a watcher, a readiness gate, an ARDI round-close — then calls a PR terminal while a reviewer is still running, which is precisely the state this criterion exists to catch.

```
gh api --paginate "repos/<owner>/<repo>/commits/<head-sha>/check-runs?per_page=100" \
  --jq '.check_runs[] | select(.status != "completed") | "\(.name) \(.status)"'
```

`--paginate` is load-bearing, not tidiness. That endpoint returns 30 check runs per page by default, so on a head with more than 30 an unfinished run can sit on page 2 while the unpaginated query returns nothing and reads as an all-clear — reintroducing, one surface over, the exact incompleteness this block is about.

The endpoint covers check runs only, so a repo that still uses legacy commit statuses needs a second query. `gh pr checks` folds both surfaces into one rollup; the check-runs endpoint does not, so swapping one for the other can hide a pending or failing status context. Read `commits/<head-sha>/status` alongside it wherever statuses are in play, and note that its combined `state` reads `pending` when the repo posts no statuses at all, which is not a pending status:

```
gh api "repos/<owner>/<repo>/commits/<head-sha>/status" \
  --jq '{state, n: (.statuses | length)}'
```

`Morrison-Lab/ai-config` returns `{"state": "pending", "n": 0}` on every head checked here, so the caveat is about other repos rather than this one.

Why the two surfaces disagree is unexplained, so do not assert a mechanism for it. Three candidates were named and none of them was tested: whether `gh pr checks` filters by check-suite app, whether it reflects only the required or branch-protection set, and whether an `in_progress` app check is omitted until it completes. The counts in the #1056 case record happen to embarrass all three, which is a reason not to adopt any of

them rather than a reason to keep looking: naming a mechanism that has survived one round of disconfirmation is still guessing, and it is the exact failure several later sections of this file are about. What is measured is the disagreement, and that alone decides which surface to read.

- **Do:** take the check-run half of criterion 1 from the paginated check-runs endpoint, and add `commits/<sha>/status` where the repo uses commit statuses, rather than treating either query as sufficient alone.
- **Do:** report both counts when the endpoint and the rollup disagree, so the gap stays visible to whoever reads the status next.
- **Don't:** read `0 pending` from `gh pr checks` as evidence that nothing is still running.
- **Don't:** drop `--paginate` — an unfinished run on page 2 returns the same empty result as a finished head.
- **Don't:** offer a reason for the omission — none was established.

Every subsection above explains a check list that is short for a per-PR reason, and a platform outage produces the same shape for a reason none of them can reach. When a repo's normal workflows never start at all, each affected PR reports a near-empty check list and `mergeStateStatus: BLOCKED`, with nothing red to point at — so the per-PR readings above all fit, and all of them send you to the wrong place. The discriminator is scope: several unrelated PRs truncated at once, plus a repo-wide `gh run list` showing a workflow type that used to run and now does not. `memories/github-actions-outages.md`'s "Check the GitHub status page when workflows stall across several PRs at once" carries the queries and the case record; reach for it before applying any subsection above to a second PR showing the same emptiness. Its sibling section there covers the other half — what the wreckage looks like once the incident clears, and why a job that is `cancelled` with zero recorded steps is an outage casualty rather than a failure to debug.

2. **The latest review is totally clean:** no nits, and every item that wasn't directly **Addressed** is either **Deferred** to a tracked follow-up issue, or **Rebutted with a rebuttal that actually convinced the reviewer** — i.e. the reviewer did *not* re-raise it on the next round. A rebuttal the reviewer still disputes does **not** count as clean. That review must be a genuine posted verdict at the current head commit, from an external reviewer if one is reachable — self-review is a fallback for when no working external reviewer is available, never a substitute once one is (see the `ardi` skill's step 2 for the availability-recheck procedure). **Pushing fixes for a finding-bearing review starts a new review cycle.** The ARDI loop is **NEVER** finished when you push fixes for a review or post an ARD disposition summary. You must wait for the new review run evaluating your latest pushed commit to post, fetch and parse that review, and confirm it contains zero findings before declaring the PR clean or ending the loop. Re-check availability right before declaring clean, not just at whichever round self-review first started; an inferred "probably clean" from green CI and resolved threads does not satisfy this.

Criterion 2's test is the absence of findings, not the presence of a verdict line saying so. A reviewer routinely asserts both at once: a `### Verdict` reading **Ready for merge**, and directly beneath it a findings section listing items nobody has addressed. Neither half is wrong, which is what separates this from the eight numbered cases in `review-verdict-pitfalls.md` — those are all a reviewer producing an unreliable or absent signal, whereas here the comment is accurate throughout and the defect is in the reading. The verdict line answers a narrower question than the one criterion 2 asks, and it is the part that appears first and gets quoted into a status report.

So when the two disagree inside one comment, **the findings win**. Read to the end of the comment before calling anything clean, and count the items under every heading, whatever that heading is called — `address-every-comment` already establishes that "non-blocking", "nit", "minor", and "optional" are prioritization labels rather than a pass, and a reviewer files findings under exactly those words in the section that contradicts its own verdict line.

The disagreement is measurable, and it is not a wording problem. Across 38 verdict-bearing `claude-review` comments sampled from 16 PRs, 8 (21%) carried a verdict line that disagreed with the findings in the same comment. Six read a pass over unaddressed nits, and two ran the other way, blocking over findings the reviewer itself called non-blocking, so the error is not a consistent bias that an offset could correct. The vocabulary is nearly closed by contrast: five outcome lexemes across five markup carriers, with 37 of the 38 naming "Verdict" somewhere. So neither detection nor parsing is the weak link.

That is the argument against gating on a machine-readable verdict field. Adding one would encode the reviewer's own looser threshold, making roughly one review in five confidently wrong in exactly the form that invites automation. Structured review output should carry **finding counts**, which are checkable against the inline-comment and thread lists, rather than a pass/fail mood, which is checkable against nothing.

A reviewer's own verification block can be wrong while its verdict is right. The verdict-versus-findings test above needs a disagreement to spot. This one offers none: the verdict is right, the findings section is empty and correctly so, and the defect sits in the arithmetic the reviewer posts to show its work.

A block labelled "verification" is the part of a review *least* likely to be re-checked, because it presents as the checking already having been done. That is what makes a wrong one worse than no block at all. It will usually sum, too, since a balancing partition is what the reviewer was aiming for, so only the composition is wrong.

Re-derive the groups rather than the total. Arriving at the right number says nothing about which groups the parts came from, and that is exactly the error a table that balances conceals.

Read it as the mirror of `ardi`'s "A systematic audit done by skimming is worse than the one-at-a-time version it replaces". That entry governs an audit *you* produce; this one governs an audit arriving *as evidence*.

The secondary signal is worth acting on rather than merely noting. A reviewer's reconstruction error usually traces to something genuinely ambiguous in the diff, so treat it as evidence about your own prose and not only about the reviewer.

- **Do:** re-derive a posted verification's groups, not just its total.
- **Do:** fix the wording that invited a wrong reconstruction, even when nothing in the diff was false.
- **Don't:** let the word "verification" stand in for having verified.
- **Don't:** read a table that sums as one that partitions correctly.

A clean verdict can ratify an enumeration instead of testing it, and then it reads as independent corroboration of a false scope claim. The entry above is the reviewer's own arithmetic going wrong. This one contains no arithmetic error at all: every member the reviewer checked was real, described accurately, and correctly called safe. It took the *set* from the diff rather than deriving it, so it verified the members that were named and never asked whether the naming was complete.

That leaves the claim worse off than if nobody had looked. An unchecked enumeration is merely unsupported, while one a reviewer has restated in its own words now carries a second signature, and the thread records the scope as confirmed by someone independent. The verdict is not evidence of independence on that point, because the reviewer's population came from the author.

The tell sits in the review's own account of what it did. A sentence naming the members it verified is reporting a check of the *cited* set, which is a different claim from the one the diff makes. So read any verdict that quotes your own count back to you as leaving exactly that count unconfirmed.

The remedy belongs in the diff rather than in the review round, because no reviewer can supply it: publish the command that derives the set instead of the count it returned, so the next reader re-derives rather than inherits. That is `avoid-hardcoding-external-data`'s prose-enumeration rule, and it is also what keeps the claim true when the next member is added.

This is the mirror of `address-every-comment`'s "a reviewer who enumerates the sites is the reason the scope goes unquestioned", and the direction is what changes the cost. There the author inherits the reviewer's list, and the failure surfaces one round later in a site the enumeration missed. Here the reviewer inherits the author's list, and nothing surfaces at all — the verdict is clean, so the loop ends. `derive-dont-enumerate` is the general principle behind both.

- **Do:** derive any enumeration you publish with a command, and publish the command beside it.
- **Do:** treat a reviewer restating your count as that count still being unverified.

- **Don't:** read a clean verdict as evidence that a scope claim in the diff is complete — a reviewer can only check the members you named.
- **Don't:** count a reviewer's agreement as independent when its population came from your own prose.

What "an approving review" means here is not a review state. Across the 25 most recent merged PRs, all 106 posted reviews are `COMMENTED` and none is `APPROVED` — `d-morrison`'s own included, so this is not a bot limitation:

```
gh api graphql -f query='{search(query:"repo:Morrison-Lab/ai-config is:pr is:merged", type:IS
  --jq '[.data.search.nodes[].reviews.nodes[].state] | group_by(.) | map({state: .[0], n: le
#=> [{"n":106,"state":"COMMENTED"}]
```

The key order there is not a typo: `gh api --jq` marshals through Go and sorts keys alphabetically, so `n` precedes `state` even though the expression builds `state` first. Plain `jq` would preserve the insertion order and print `{"state":...,"n":...}`.

A constant carries no information, so `.state` cannot confirm clean here, and waiting for a formal `APPROVED` would stall every PR indefinitely. Approval is established instead by the two reads criterion 2 and the **Threads** paragraph already name: zero findings in the latest review body, and zero unresolved inline threads. The one state that does still carry information is `CHANGES_REQUESTED`, which stays blocking however a later verdict line reads.

- **Do:** read the whole review comment and count findings under every heading before calling a PR clean.
- **Do:** establish approval from the findings and thread lists, since `.state` is `COMMENTED` on every review this repo receives.
- **Don't:** quote a **Ready for merge** line as the clean signal while the same comment lists findings.
- **Don't:** wait for a formal `APPROVED` review, or read `COMMENTED` as a defect in the reviewer.

Findings hide on several surfaces, and no single check sees all of them — so read the verdict body, any suppressed-comments block, the inline comments, the thread list, and the verdict's own conclusion every round. The entry above is about a reviewer contradicting itself inside one comment. This is about the *detection method* returning an answer that is technically true and substantively wrong, which is harder to notice because nothing looks inconsistent.

- **An out-of-diff finding never becomes a thread.** A finding about a line the diff did not touch cannot be attached as an inline comment, so it appears only in the body — reviewers say so explicitly ("inline comments were unavailable for out-of-diff lines"). A thread count therefore cannot see it. Zero unresolved threads is not evidence of zero findings.

- **An empty body hides the mirror case.** A review can post a completely empty top-level body and carry its entire finding in one inline comment, so a body-only read finds nothing to act on and concludes there is nothing.
- **A clean overview can hide a collapsed findings block.** Copilot can say it "generated no new comments" and create zero inline comments while placing substantive findings inside a collapsed `<details>` suppression block in the review body. The heading moves, so match case-insensitively on `suppressed` **inside the** `<summary>` **heading**, not anywhere in the body: PR #660 emitted `Comments suppressed due to low confidence (3)`, while PRs #1029 and #1031 emitted `Suppressed comments (4)`. A literal grep for either exact phrase can return a false zero. A body-wide match over-corrects the other way and can permanently reject a genuinely clean review, since ordinary overview prose can also contain the word — review 4837572117's summary table read "suppressed Copilot findings" outside any collapsed block. A body read that stops at the overview is therefore not a body read, and a match against the whole body is not the right instrument either.
- **"No verdict" is its own state, distinct from "a verdict with no findings".** A review job can fail having posted *nothing* — not a stub, not an empty comment. Zero findings and zero review are indistinguishable by any count, and they call for opposite responses: one is done, the other needs a self-review and a re-run. Read the job's step outcomes when a review is missing rather than inferring from the absence of comments.

The reason this defeats otherwise-good instruments is that each check answers a narrower question than the one being asked. "Are all threads resolved" is not "are there no findings", and neither is "does the verdict say ready". Per `algorithmatize-checks`, prefer the instrument that decides the question exactly — and where none does, as here, say so rather than substituting the nearest available count.

- **Do:** read all review surfaces before calling a PR clean, every round, including collapsed suppressed-comments blocks.
- **Do:** distinguish "no findings" from "no verdict" explicitly, and treat the latter as unreviewed.
- **Don't:** report clean on a zero thread count, however many checks are green.
- **Don't:** treat an empty review body as an all-clear without checking the inline comments.
- **Don't:** treat a "generated no new comments" overview as an all-clear until every `<summary>` heading has been checked case-insensitively for `suppressed` — not until the whole body has, which flags ordinary overview prose that merely mentions suppressed findings.
- **Don't:** read a reviewer's silence as a verdict — a job that posted nothing leaves the same zero counts as a job that found nothing.

A comment can be evidence-dense, correct throughout, and state no verdict at all — and its density is what gets read as the conclusion. The "no verdict is its own state" bullet above covers a job that posted *nothing*, and the instrument it prescribes is to read the job's own step outcomes. Neither half reaches this case. The job succeeded, the comment is

long and rigorous, and there is no failed step to inspect — so that remedy points at a surface reporting success.

It is not the "reviewer's own verification block can be wrong" case either, which is a *wrong* verification under a *right* verdict. Here the verification is correct and there is no verdict at all, which inverts which part deserves suspicion. That section already notes a block labelled "verification" is the part least likely to be re-checked, because it presents as the checking having been done. When such a block is the last thing on the thread it does something further: it reads as the sign-off, and the more rigorous it is the more it reads that way. So thoroughness is not evidence of a conclusion — it is what disguises the absence of one.

A later comment stating no verdict does not supersede an earlier one. This refines the latest-wins rules rather than contradicting them. Those rules (`CLAUDE.md`'s "re-read the **most recent** review comment", and criterion 2's "latest review") assume the most recent artifact *is* a verdict, and say to prefer it over a cached one. They do not say what happens when the most recent artifact concludes nothing. Absence is not a clearing: the standing verdict is the last one anyone actually stated, however much has been posted since. Read "latest" as ranging over verdict-bearing statements, not over comments.

Note this is wider than the HEAD-SHA scope the rest of criterion 2 uses. A "Needs more work" posted against an *earlier* commit is outside every HEAD-matching check, and a later verdict-less comment raises no finding either, so a PR reads clean on both while its last real verdict was not. `scripts/check-pr-fully-clean.py` decides this as its criterion 4, scanning the whole review history chronologically for the last verdict-bearing statement.

- **Do:** identify the last statement that actually states a verdict, and treat that as the standing one.
- **Do:** scan the whole review history for it, not only items matching HEAD.
- **Don't:** read a verification section, however rigorous, as an approval — it is evidence, and a verdict is a conclusion about evidence.
- **Don't:** treat a later comment's silence on the verdict as superseding an earlier "Needs more work".

(Morrison-Lab/ai-config#1267, 2026-08-07, reverted by #1275. Verified from the API rather than from the revert's own account: the PR carried `reviews | length` of 0, and its four comments ran `21:56:09Z` **Needs more work**, `22:12:47Z` no verdict, `22:49:12Z` **Needs more work**, `23:05:32Z` no verdict — a long `### Verification` section ending "Not merging." It was merged at `23:38:12Z`, 49 minutes after the last stated verdict, and reverted at `23:47:50Z`. All four comments were posted under the author's own login, so "the PR has been reviewed" was true while "an independent reviewer approved it" was not.)

Another surface, and the one that defeats the gate itself: the review check can pass on a blocking verdict. The cases above are ones where a *reader* looks at the wrong place. This is the case where the repo's own gate looks at the right place and still reports

green, because `require-review` tests whether a review **ran**, not what it **concluded**. So a "Needs more work" verdict and a "Ready for merge" verdict produce an identical check row.

It compounds with case 1 in `review-verdict-pitfalls.md` rather than sitting beside it. A review invoked without a `--comment` argument reports its findings in the run's own comment and posts nothing as a thread — and the better reviewers say so in their last line, which is the tell worth grepping for. The result is a PR with every check green, zero inline comments, zero unresolved threads, and a blocking correctness finding sitting in plain text that no count reaches.

This is the third numbered case in `review-verdict-pitfalls.md` – a check that cannot fail on its own content, so its green carries no signal – arriving on the one job whose whole purpose is to gate on review outcome. The difference is what makes it worse than the benchmark check recorded there. That one is *designed* never to block, and a reader who knows the design knows to read its comment. `require-review` is designed to block, is frequently a required check, and still reports green on a verdict that says the opposite. Read the verdict line itself, every round; a green `require-review` is evidence a reviewer spoke, and nothing more.

- **Do:** grep the verdict body for its own conclusion, and treat a `require-review` pass as orthogonal to whether the PR is clean.
- **Don't:** let a green review-gate check stand in for reading what the review said.

`check-pr-fully-clean.py` itself has the mirror false positive: it can report NOT clean over a clean verdict. The cases above are fail-open — the instrument reads clean when the PR is not; this script fails the other way. Its `finding_patterns` scan ran over the whole review body, so a clean `Ready for merge` verdict that merely *quotes* finding vocabulary — `**Location:**`, `Needs more work`, and the like — tripped a pattern and printed `contains findings (matched pattern ...)`. It bites hardest on PRs about the review tooling itself, whose reviews naturally discuss finding-indicator words, and its direction is fail-closed, so it is the safe one: it makes the script untrustworthy for auto-confirming clean, never for waving a real finding through. The scan now blanks cited finding vocabulary — fenced code blocks, inline code spans, and double-quoted spans — before matching (Morrison-Lab/ai-config#1202), so the two documented instances (a `**Location:**` code span, a double-quoted `Needs more work`) no longer trip it, while the structural findings-heading and formal `CHANGES_REQUESTED`/`REJECTED` checks remain as independent backstops. A finding-mood phrase stated *unquoted* in prose, or in a blockquote line the strip does not cover, can still trip it, so when the script does flag on quoted vocabulary the remedy is unchanged — read the verdict's own conclusion rather than the script's raw pattern match.

- **Do:** read the verdict's own conclusion when the script reports findings against a review whose prose merely discusses finding vocabulary.
- **Don't:** treat a `contains findings (matched pattern ...)` line as a real finding without reading the verdict body it matched.

A verdict comment quotes verdict phrases, so a phrase search identifies nothing — and it misreads in both directions at once. Every case above concerns *reading* a verdict correctly. This one is about the instrument a multi-PR status sweep reaches for, where the tempting shortcut is a one-line `jq capture` of the first verdict phrase appearing anywhere in the body.

The premise under that shortcut is that a verdict comment states verdict vocabulary only when stating its own verdict. It does the opposite. Quoting is part of the genre: a comment cites the previous round's verdict to say what it is confirming, pastes a repro block showing what a classifier returned, and discusses what a phrase *should* classify as — all before reaching its own `### Verdict` section. So the first match is usually somebody else's verdict.

The bidirectionality is what makes this worth its own entry rather than a note on the section above. That one is fail-closed by construction, which is why it is called the safe direction. A first-match phrase search has no direction at all, so it cannot be corrected by an offset or by assuming the reviewer errs one way. Measured on one sweep, 2026-08-08, taking the latest verdict-bearing comment on each PR:

PR	first phrase match	real verdict, at the last `### Verdict`	direction
#1278	`Ready for merge`, inside a fenced block quoting a classifier call	**Needs more work**	false-clean
#1257	`Needs more work`, inside a parenthetical citing the prior round	**Ready for merge**	false-blocked

The false-clean direction is the expensive one: it produced a **merge recommendation** on a PR whose verdict was blocking.

So call the instrument. `scripts/check-pr-fully-clean.py` is this corpus's verdict authority, and `ardi` already requires it for the single-PR loop — the gap is that nothing said so for a **sweep**, which is where the hand-rolled parser goes in. That is `deterministic-tools`'s constraint violated in the presence of the instrument, which is the shape worth recognizing: the tool existed, was documented, and was mandated one workflow over.

Where a body genuinely must be parsed by hand, anchor on the **last** `### Verdict` heading and take the first non-empty line after it, which returns the right answer on both rows above. Two hazards survive even then, and both were observed on #1278. A `### Verdict:` heading can itself appear quoted inside prose, so the *last* heading rather than the first is load-bearing. And a **human** comment can carry a backticked `### Verdict` while stating no verdict at all, so select candidates on the `**Claude finished` body marker above rather than on the presence of a heading.

- **Do:** call `check-pr-fully-clean.py` for a sweep's verdict column, exactly as `ardi` requires for one PR.
- **Do:** anchor on the last `### Verdict` heading when parsing by hand, after selecting candidates on the `**Claude finished` marker.
- **Don't:** take the first verdict phrase in a body as that body's verdict — quoting other verdicts is part of what a review comment does.
- **Don't:** assume such a misread has a safe direction; one sweep produced a false-clean and a false-blocked.

A review comment's header SHA can be stale, so take the reviewed commit from the run's own `head_sha`. Criterion 2 requires the verdict to sit at the current head, and the obvious instrument for checking that is the unreliable one: the commit named in the comment's own caption. A verdict captioned with a superseded commit can be a current-head review whose caption simply names a different commit than the run checked out.

The failure direction is the expensive one. It reads as a stale review, which invites a needless re-trigger, and under `concurrency: cancel-in-progress` that re-trigger cancels the run already in flight at the real head. So the caption costs you the verdict it was making you doubt.

The run's `head_sha` settles it, and the comment links the run it came from, so the check is one call. This is `algorithmatize-checks` applied to a verdict: prefer the API field over the prose caption.

- **Do:** follow the job link in the comment and read that run's `head_sha`.
- **Don't:** treat the SHA in a comment's heading as the commit reviewed.

That remedy assumes the run checked out the PR head, and a `workflow_dispatch`-triggered review run does not. `claude-review.yml` dispatched with a `pr_number` input runs against `ref: main` — its own `head_sha` is whatever `main`'s tip was at dispatch time, not the PR branch or the commit its gather-context step actually diffed. The job fetches the PR's diff separately, through the API, inside the run, so nothing in the run object records which PR commit that fetch saw. Reading `head_sha` here answers a different question than the one being asked, and it answers confidently: a real SHA, on a real branch, that happens to be irrelevant.

So for a `workflow_dispatch` run, the SHA check has no target to read. Fall back to **timing**: compare the run's `created_at` against your own push timestamps. A run dispatched before your latest push cannot have reviewed it, whatever its verdict claims about "the current diff." Where the verdict makes a specific claim ("this wording is unchanged"), the cheapest confirmation is direct: read the file yourself and check whether the claim is still true. A verdict that is empirically wrong about present file content is conclusive proof it reviewed an earlier one, with no run metadata needed at all.

- **Do:** check a `workflow_dispatch` review's `event` field before reaching for `head_sha` — on that trigger type the field names the dispatch ref, not the reviewed commit.

- **Do:** cross-check a stale-suspected verdict's specific claims against the file directly, rather than only against run metadata.
- **Don't:** trust `head_sha` as "the commit reviewed" on a workflow-dispatch-triggered run — that guarantee only holds for push/pull_request-triggered runs, which check out the PR head by construction.

A clean CI run and a clean review verdict are a snapshot, not a standing guarantee of mergeability. `main` can advance after your last check — including gaining its own independent addition that collides with yours (see `sync-with-main.md`'s "two PRs append the same numbered subsection" case) — so re-verify the branch still merges cleanly against current `main` before reporting a PR ready, not just trust the last green run.

Re-check version parity in that same sweep, not only conflict-freedom. sync-with-main already covers comparing `DESCRIPTION` versions *after merging* ***`main`*** *in*. The case that rule misses is the one with no merge at all: `main` advances on its own after your last review round and lands on the branch's exact version, so an R package's `version-check` job (which requires the branch to *exceed* `main`) goes from green to red with nothing to point at. There is no conflict, no failing check yet, and no warning — the last run passed because `main` was still a version behind when it ran. So the declare-ready sweep needs both `git merge-tree` for conflicts and a direct version comparison; either one alone reports a PR ready that isn't.

Threads: at fully-clean, every **inline** review thread is resolved, and the only conversation left open is the final all-clear exchange — the reviewer's all-clear comment and your reply to it. (The all-clear is usually a top-level PR comment, not an inline thread.) Check this mechanically rather than from a memory of which threads you replied to. Which field name to look for depends on the surface: the GitHub MCP tool `pull_request_read get_review_comments` returns thread objects under a `review_threads` key with snake_case `is_resolved`/`is_outdated`, while a raw `gh api graphql reviewThreads` query — what resolve-pr-threads, `pr-status`, and `ard` use — returns camelCase `isResolved`/`isOutdated`. Both are correct on their own surface; this is the same REST-vs-GraphQL casing split the check-state paragraph above already warns about, so read the response you actually get rather than assuming one spelling. Either way, sweeping for the unresolved ones is the entire check. An **outdated** thread (`is_outdated: true` — the code it anchored to has since changed) still counts as unresolved: addressing a finding and resolving its thread are separate actions, and only the second clears this criterion. An addressed-but-unresolved thread reads as outstanding work to every later reviewer, which is exactly what this criterion exists to prevent.

One finding can own two threads, so sweep by thread id rather than by finding. When a reviewer re-raises an item you already answered, the re-raise often opens a **new** thread instead of continuing the original — same file, same line, same finding, different `threadId`. Resolving the one you remember replying to therefore leaves a second thread behind, and it is easy to miss twice over: it is usually marked `is_outdated: true` (the line it anchored to has since changed), and your own memory of the exchange says the item was settled. Neither of those clears it. Re-read the thread list before declaring clean and resolve every entry whose

`is_resolved` is false, whatever you recall about the finding it carries; reply on the second thread too, pointing at the first, so a reader landing on either one sees the resolution.

Deadlock -> escalate to a human. If you and the reviewer(s) can't reach consensus on an item (a rebuttal was exchanged and neither side is budging), don't loop forever and don't unilaterally override the reviewer — request a **human reviewer**, `@`-mention them in a comment summarizing the impasse, and surface the open item.

An automated reviewer's verdict on a disputed factual/technical claim is not stable across independent runs, even with identical evidence available each time. Don't treat one round's "settled, no need to keep arguing" as durable: the very same review job, re-triggered later with no new code changes, can re-raise a claim it previously retracted — and then retract it again on a subsequent run — purely from re-deriving the question differently each time, not from anything changing in the PR. This means a rebuttal thread's outcome (however many rounds of citations and counter-citations) doesn't itself resolve a genuine deadlock the way a human's decision does; only escalating per the bullet above actually settles it. The one thing that DOES help going forward: fold the authoritative citation/evidence directly into the code or doc being reviewed (a comment, not just a PR conversation reply) — a fresh reviewer run re-deriving the claim from scratch is more likely to find the citation sitting right next to what it's evaluating than to dig through prior thread history for it, though even that is not a guarantee against a bot that ignores context already in front of it.

The several distinct ways a review job's check color, or the presence and content of a posted comment, can diverge from a genuine, complete, correct verdict — the "eight numbered cases" this file used to walk through inline — now live in `review-verdict-pitfalls.md`, split out per ai-config#1236 once that material pushed this file past the size gate.

Address Every In-Scope Review Comment

When iterating on a PR with a reviewer, **address every in-scope flagged item**, regardless of severity label. The reviewer's "Not a blocker", "minor", "nit", "optional", "consider", or "if you want" labels are for prioritization, not a free pass for the implementer.

Worked-example case records for the rules below live in `address-every-comment.cases.md`, moved out of the auto-loaded context.

For each flagged item, do exactly one of:

1. **Fix it in this PR.** The default path — most nits are 1–3 line changes.
2. **Defer.** Only when the fix expands the PR's scope (new feature, broader refactor, separate concern), the requester has explicitly said this PR shouldn't grow, or the flagged content isn't actually yours to fix here (see the `main`-sync case below). File a follow-up issue and reference it in a PR comment so the item isn't lost — except in the `main`-sync case, where the "follow-up" is fixing it on `main` directly, not a new issue.

Then trigger another review and repeat until the PR is **fully clean** — zero flagged items under any heading, no "non-blocking", "harmless", "minor observation", or "could improve" sections. "Looks good" / "no findings" / "approved" with no follow-on bullets is the bar. Resolve every inline review thread along the way, leaving only the final all-clear exchange.

Always resolve an inline thread the moment its comment is successfully addressed — the fix pushed and a reply posted naming it — in the same pass, whatever workflow you're in: a formal `ard`/`ardi` round, a CI-monitor nudge, or a one-off fix outside any loop. Addressing without resolving leaves a thread that reads as outstanding work to every later reviewer, blocks `fully-clean`'s every-inline-thread-resolved criterion, and drags stale noise into the next review round. The per-disposition settlement rules in `ard` step 4b still govern the exceptions: a **Rebut** stays open until the reviewer drops it, and an **Address** you're not confident fully settles the concern gets a reply asking for confirmation instead of a resolve. The `resolve-pr-threads` skill sweeps any stragglers, but it's a backstop — resolve-on-address is the default, not a cleanup step.

Do **not** report "ready to merge with one minor nit noted" / "harmless as-is" / "can address if you want" — that hedging just pushes triage back to the requester.

A round count is never a reason to stop, and "the reviewer keeps finding things" is not a finding about the reviewer. There is no threshold after which unaddressed items become acceptable: keep requesting reviews and keep dispositioning findings until a review comes back with none. The only exits are a totally clean review, a genuine per-item deadlock, or the user calling it. Reasoning of the form "we have done N rounds, shall we accept the current state?" is the same hedging this paragraph bans, moved up from one finding to the whole loop — see `ardi`'s "Stopping conditions" for why it fails and for the case record.

Noise is per-item, not per-round — don't stop the whole loop over one recurring flag. A long-running PR can have both real findings (worth fixing every round) and one specific item the reviewer re-raises verbatim round after round even though it's already deferred/tracked (e.g. a file-length guideline already split into a follow-up issue). Keep fixing every *new* finding as it appears — don't let the recurring item make you stop processing genuinely new ones. But stop re-litigating *that one item* every round: reply once pointing at the tracked issue, and hold on it specifically rather than re-deferring it on each pass. Surface the pattern to the user (which item, how many rounds, where it's tracked) and let them decide whether to resolve it now (e.g. do the split) or leave it as accepted recurring noise — don't decide unilaterally to either keep re-processing it or silently drop it.

When a finding is a pattern (a formatting/style rule broken in one spot), apply it everywhere it recurs in the same file, not just the flagged line. A reviewer that flags one inconsistent list-item format is telling you about the rule, not just that one item — fix every occurrence in the same file that breaks it in the same pass, rather than waiting for the reviewer to flag each occurrence in a separate round. Re-scan the whole changed file for the same pattern before pushing the fix.

That rule's scope is "the same file", and a reviewer who enumerates the sites is the reason the scope goes unquestioned. A pattern finding usually arrives with a list attached: three spots, named, each with a file and a line. That list is a snapshot of where the reviewer happened to look, never the extent of the pattern. A reviewer reports the instances it noticed while reading a diff, which is not the same operation as sweeping for them.

Inheriting the list rather than deriving it reads as responsive rather than as under-scoped, which is what lets it survive a round. The reviewer found the problem, so its account of the problem carries the authority of the finding itself, and fixing exactly what was named is indistinguishable — from the inside and from the thread — from fixing the pattern. The failure then reproduces one round later in a file the enumeration did not name, which is the round the rule above exists to save.

The tell is lexical, and it sits in your own reply: **"fixed all three spots you named"**. Quoting the reviewer's count is the signal that the set was inherited, since a derived set has no reason to agree with a number someone else supplied.

So derive the site list with a command: pipe the whole diff through a grep for the flagged phrase, and widen the search past the diff when that phrase was copied in from somewhere else.

Pick the pattern shorter than the phrase, though. This corpus mandates semantic line breaks, so a multi-word phrase routinely straddles a newline, and a line-oriented grep for the whole thing then matches nothing — a derived set can come back empty for a formatting reason rather than a factual one, which is the same false-negative this file treats at length under its own semantic-line-break corollary. A short distinctive fragment is the reliable choice.

Then report what was **swept** rather than what was fixed. "Grepped the whole diff for `X|Y|Z`, four hits, all four fixed" is checkable, while "fixed all three" asserts a scope nobody measured.

This is `derive-dont-enumerate`'s principle applied to a review finding's site list rather than to work items, and that is the transferable part. Every fix is individually correct while the coverage claim over them is false, so the gap is a property of the **set** rather than of any member — nothing in the diff, the tests, or the thread reports it.

Distinct from `algorithmatize-checks`'s "never predict which case will fail; enumerate the class", which shares this remedy and has a different trigger. There the list is one **you** produced from intuition, so the rule fires on your own naming of a member. Here the list arrived from someone with more standing to write it than you had, which is why nothing about accepting it feels like guessing.

- **Do:** derive the site list by grepping the whole diff for the flagged phrase, and fix what the grep returns.
- **Do:** report the sweep — the pattern searched and the hit count — rather than the number of sites you fixed.

- **Don't:** treat a reviewer's enumeration as the extent of the pattern; it is the extent of that reviewer's read.
- **Don't:** write "all N spots you named" into a reply, since quoting the reviewer's count is the tell that no sweep ran.
- **Don't:** read a null result as "no further sites"; it means no further hit for that pattern, and a differently-worded instance would not have matched.

Deriving the class is necessary and not sufficient, because you can derive the wrong one — and the growth rate across rounds is what says so. Everything above governs inheriting a reviewer's list instead of deriving your own. This is the failure that follows taking that advice. You derive a class, extend the enumeration well past the reported members, and report the sweep exactly as those bullets ask — and the next round returns more members, because the class you enumerated is not the one the defect lives in.

Note what that does to the tell. The lexical one above catches a reply quoting the reviewer's count, and a reply reading "I derived the class rather than fixing the N reported instances" passes it cleanly while being wrong in the same way. Its replacement is quantitative and needs no insight: **if round 1 grew the list by N and round 2 produces M more, the list is not one pass from done.** The growth is itself the evidence that enumeration is the wrong lever, and it is sitting in front of you the moment round 2 lands.

The reason the diagnosis loses is structural rather than careless. A review that finds this usually supplies a diagnosis *and* example cases. The examples are actionable and the diagnosis is not, so the examples get worked and the sentence gets read past. Worse, such a review will often pair its diagnosis with a suggested fix that is itself another enumeration step — so the actionable half points at the very lever the diagnostic half just warned about, and following the review faithfully reproduces the bug.

The way out is a lever on a **different axis**, picked by measurement rather than by taste. Ask what the enumeration is actually protecting against, and whether that is the same property the enumeration is expressed in. When it is not, the fix is solving one problem with another problem's instrument, which is why it keeps costing rounds. The measurement is cheap: relax the enumeration entirely and read which cases change. If the cases that move share some other property, that property is the axis the fix belongs on. A useful confirmation that the class is closed rather than the members patched is that most of the resulting test cases were never reported by anyone.

- **Do:** read growth in the list across rounds as evidence about the lever, not as a count of members still to add.
- **Do:** state a reviewer's diagnosis back in your own words before acting on its examples, so a redirect cannot be worked past in silence.
- **Do:** relax the enumeration and read which cases move — a property shared among them names the axis.
- **Don't:** treat "I derived the class rather than fixing the reported instances" as discharging the rule above; it is that claim one level up, and it fails the same way.

- **Don't:** prefer the actionable half of a review to the diagnostic half merely because it is the half you can start on.

A narrower version of the same failure: the class is right, and it is enumerated in more than one place. The block above governs deriving the wrong class. This is what happens once the class is right — you fix the site the round reported, the concept turns out to live at two or three sites, and the next round arrives through one of the others. Each round then feels like a new finding while being the same room entered by a different door.

The tell is that consecutive findings paraphrase to one sentence. When three rounds all reduce to "text handed to something that runs it", the recurrence is not about a class's members but about the number of **places that class is written down**. So the quantity to derive is the site count: grep for the concept rather than for the construct that exposed it, and expect the review's own prose to have named the sibling site already — ours did, observing that one list "already enumerates programs whose quoted argument is live" while the site that failed had "no analogous carve-out".

The fix is DRY rather than another member: define the concept once and have every site consume it, so the residual is a single reviewable list instead of several that drift apart. That converts an unbounded sequence of rounds into one artifact a reviewer can check, which is the only form of "this is closed now" worth claiming. See `dont-incur-technical-debt` for why the second copy was the defect rather than the newest gap in it.

- **Do:** paraphrase the last two or three findings into a single sentence, and read a match as evidence that a concept is duplicated rather than incomplete.
- **Do:** derive how many sites encode the concept, then consolidate them into one definition every site consumes.
- **Don't:** answer a third instance by extending a third list — that is the same round again with a new door.
- **Don't:** skip the review's own prose naming a sibling site; it is frequently there, in the paragraph explaining why some other mechanism did not save you.

The mirror case: the enumeration was complete and the fix was not. The rule above governs a reviewer's list that was too short. This governs the one that was exactly right, and a reply that closed it anyway. A finding naming two artifacts — a stale equation *and* the prose describing it, a constant *and* the comment above it — is two findings sharing one comment, so it earns two dispositions rather than one. Fixing the first and writing "Addressed" under-delivers against a list nobody had to derive, which is why no sweep rule catches it: the grep the section above prescribes was never the missing step.

What makes it survive the round is that the source diff looks finished. The flagged line is visibly changed, the commit message names the finding, and a reviewer re-reading the diff sees a real fix where the finding pointed. The unfixed half is *context* in that diff rather than a hunk, so nothing about reading the diff distinguishes "both halves done" from "one half done".

So verify a prose or formula fix against the **rendered** artifact wherever the project builds one — a PR-preview deploy, a `gh-pages` build, generated docs. The rendered page puts both halves of a finding in one view, in the order a reader meets them, which the diff never does. `fact-check-prose`'s rendered-artifacts bullet owns the mechanics of locating that preview; the increment here is *when* to reach for it — closing out a finding, not only checking a computed figure.

- **Do:** count the artifacts a single comment names, and give each one its own disposition before replying.
- **Do:** read the rendered page rather than the diff when confirming that a prose or formula fix landed completely.
- **Do:** grep the whole file for the underlying concept once a second half surfaces — a document stale in two places is usually stale in three.
- **Don't:** let a visibly-changed flagged line stand in for the finding being closed; the unfixed half appears in the diff as context.
- **Don't:** reach for the derive-the-site-list remedy above here — that list was complete, and the shortfall was in the delivery.

When a prose fix changes wording that's also paraphrased elsewhere in the same PR (a CHANGELOG entry, a PR description, a cross-reference), sync that copy too. A CHANGELOG entry written before the review lands often quotes or paraphrases the exact phrase a reviewer later flags; fixing the source prose but leaving the paraphrase stale reintroduces the same wording issue one file over. Grep the diff for the flagged phrase before considering the finding closed.

When syncing copies, search the diff for the claim, not the files or symptom already in front of you. The paragraph above says to grep the diff, and both words matter. A path-scoped grep over the files already open is not a diff search, however real the hits it returns are. The scope is the silent variable: the command succeeded, but it searched the author's working memory rather than the change. Pipe the diff itself, so the search space is the whole PR diff:

```
git diff origin/main...HEAD | grep -n "<figure-or-phrase>"
```

Run that after committing, not before. Like `check-new-line-breaks`, any `origin/main...HEAD` scan reports on `HEAD`, so a pre-commit run describes the old committed text and can make a fixed working tree look unfixed.

The same failure can hide in the search term instead of the path. When a review retires a rationale, search for statements of the retired criterion, not only for the word or contradiction that exposed it. The exposing detail usually appears once; the criterion is what got copied around.

- **Do:** run whole-diff searches for synchronized figures and phrases, after committing the fix, and report the before/after counts.
- **Do:** when a rationale is retired, search for every wording that states that rationale or criterion, not only for the symptom word that made it fail.
- **Don't:** substitute `grep -rn <term> <files-you-had-open>` for grepping the diff.
- **Don't:** accept a search for the visible contradiction as proof that the retired claim itself is gone.

When the wrong thing is a figure, the unit of repair is the figure — across every artifact carrying the twin, not just the diff. The rules above all scope the search to the diff, or to the same file. That is the right scope when the duplicate was created *by* this change. It is the wrong one when two artifacts have carried near-duplicate prose for a while — a test file's explanatory header and a memory file, a docstring and a design doc — and only one of them is in your diff at all. The other copy is outside every instrument above by construction, so a clean whole-diff grep is not evidence about it.

So when a finding names a wrong number, treat the **value** as the search term and the **set of artifacts known to carry the twin** as the search space. `scripts/find-near-duplicates.py` already exists for locating that twin; reach for it rather than recalling which files pair up. Reporting the sweep the way the rule above asks — the term searched and the hit count, per artifact — is what makes the coverage checkable rather than asserted.

And a reflow puts its neighbouring sentences into your change, for fact-checking and not only for lint. `ascii-punctuation-in-source` already establishes the diff-attribution half: editing a line for an unrelated reason makes its pre-existing violations yours, "because the check reads added lines, and a modified line is an added line." Its scope is mechanical — banned glyphs, multi-sentence lines — and its remedy is to bring the line into compliance. The same argument reaches further than that section claims. If reflowing a paragraph makes its em-dash yours, it makes its **wrong figure** yours too: the sentence is in your diff, and a sentence in your diff is a claim you are asserting. Re-read every sentence a reflow touched against `fact-check-prose`, not only against the punctuation checks.

- **Do:** grep for the figure's value across every artifact carrying the twin, before replying that the finding is closed.
- **Do:** fact-check the sentences a reflow pulled into your diff, exactly as you would the ones you wrote.
- **Don't:** treat the named occurrence as the unit of repair when the same value appears elsewhere.
- **Don't:** read a clean whole-diff grep as covering a twin the diff never touched.

(`Lacaedemon/sparta#1222` and `#1225`, both 2026-08-07, both touching exactly `.claude/memories/sparta.md` and `test/unit/test_residual_melee_swirl_battle.gd` — the twin pair. Three misses in one PR lineage: #1222 round 2 fixed a reconciliation in one file only; #1225 round 1 fixed the

test header and left the memory copy; and within that same PR a second wrong figure one sentence away kept its wrong attribution, in a paragraph that edit had itself reflowed.)

The PR description is on that list and is the one copy grepping the diff cannot find, so check it separately. A PR body is not a file, so it appears in no diff and no reviewer reads it as part of the change under review. That makes it the copy most likely to survive a fix, and the copy most likely to be *read* by someone deciding whether to merge — so a stale one teaches the reader exactly the thing the diff was corrected to remove.

The tell is a fix to something the PR body summarizes: a behaviour change, a mechanism, a rationale. Re-read the description against the corrected diff before declaring the round done, and say in the update that it was corrected, so a reader who saw the original knows it was revised rather than always having said this. Where the correction has history worth keeping — a claim that was wrong and is now right — state it as history in the body rather than silently overwriting, since the wrong version is what earlier comments respond to.

- **Do:** re-read the PR description after any Address that changes what the PR does or why, alongside the changelog check above.
- **Don't:** treat a clean `grep` over the diff as evidence every paraphrase is synced — the description was never in it.

Following that "state it as history" advice is what produces the next block, because an automated reviewer reads the body as a flat statement of intent. The paragraph above is right that a correction with history worth keeping should be recorded rather than silently overwritten, since earlier comments respond to the old version. It has a failure mode it does not warn about, and the failure lands precisely on the authors who follow it.

A past-tense paragraph saying a thing *was* excluded is, to a bot, not distinguishable from a claim that it *is* excluded. Tense is doing all the work, and nothing in the reviewer's reading of the document preserves it. So the more faithfully the reversal is recorded, the more confidently the reviewer reports the diff as contradicting its own description — and the remedy it proposes is to revert the change, which means undoing whatever the reversal was.

Distinguish this from an ordinary stale snapshot before answering. A reviewer that started before your edit never saw the correction and needs only a pointer to it. The timestamp check further down is written about a missed *rebuttal*, but the same `started_at` comparison decides a missed *body edit*: a body corrected after the run began is invisible to it for exactly the same reason, since the whole PR is snapshotted once at run start. This one re-raises *at the corrected text*, so the timestamps clear and the finding still stands. Compare the run's start time against the edit, then read which passage the new verdict quotes — if it is quoting your history section, this is the case, not that one.

- **Do:** state the current content first, marked as current, before any history.
- **Do:** put the reversal in its own section that opens by saying it is history.

- **Do:** make sure the "what is excluded" section does not name the reversed item at all, in any tense.
- **Don't:** rely on past tense alone to carry the distinction.
- **Don't:** revert a maintainer-requested change because a reviewer read the history as current — rebut, and escalate rather than comply.

Be honest about the residual: all of that can be applied and a further run can still block, at which point the only remaining move is deleting the history outright, which costs the earlier comments their referent. That trade belongs to the human, not to the agent driving the PR.

The same sync is needed when the review fix is to CODE BEHAVIOR rather than to wording — and that case is easier to miss, because nothing about fixing a bug points at the changelog. The rule above fires on a recognizable trigger: a reviewer quotes a phrase, so you go looking for that phrase. A behavior finding gives you no phrase to grep. You change the code, update the PR body's description of what it now does, and the `NEWS.md/CHANGELOG.md` entry — written before the review, in prose that described the *old* behavior correctly — goes on asserting it. Every later round then reviews a diff whose changelog contradicts its own code, and no reviewer flags it, since each file reads plausibly on its own. The shipped result is worse than a stale paraphrase: a user reading the release notes is told the opposite of what the release does. So after any Address that changes behavior, re-read the PR's changelog entry against the new behavior — not just the code and the PR body. Fold it into the same pre-push self-review pass `ardi` already requires; a changelog entry is a claim about the diff, so `fact-check-prose` applies to it exactly as it applies to any other prose in the PR.

Tighter still: a changelog entry can contradict its own commit message, in the same commit, with no review in the loop at all. Both cases above need a review round to set them up — a reviewer quotes a phrase, or a finding changes behaviour — so the trigger to go looking is external. Here there is none. The commit message and the changelog entry are written minutes apart, by you, in the same commit, and disagree.

The reason it survives is that the two are drafted in different registers. A commit message argues for the change and reaches for the sharpest true statement of the mechanism; a changelog entry describes the change for a release note and reaches for the tidiest one. Nobody reads them side by side afterwards. A diff review sees one, a `git log` sees the other, and no check compares them — so the contradiction ships, and the release notes are the half a user actually reads.

The check is mechanical and belongs in the pre-push self-review pass `ardi` already requires: after writing a rationale into a commit message, grep that same commit's prose changes for a claim about the same mechanism, and read the two together before pushing. Where they differ, the commit message is usually the correct one, because it was written while the mechanism was in front of you.

One step further back: a figure inherited from the tracking issue is both the copy git keeps and the copy nobody verified. The entry above explains a mismatch by *register* — a commit message argues for the change while a changelog describes it, so they get drafted differently and never read together. Here both claims sit in the same register and describe the same fact. Only one of them was checked. What separates them is **provenance**: a number produced by running something, versus a number carried over from the issue you wrote before you had anything to run.

Two properties make it worse than an ordinary wrong number.

One of the two copies becomes permanent, and you cannot tell which from inside the PR. A PR body stays editable forever, while a commit message does not survive a merge in editable form — but which text a squash merge actually keeps is a repository setting, and it can be either. Configured one way the commit messages land on `main` and the PR body is discarded; configured the other the PR body becomes the commit body and the commit messages are dropped.

That is why the rule is *both must be right* rather than *check the important one*. The copy that survives is chosen by a setting most authors have never looked at, so treating either as the draft is a coin flip. And the odds are not even: the commit message is the one written earliest, from the least evidence, so the configuration that keeps it is the one that makes the weaker copy permanent.

Read a recent squash commit on `main` if you want to know which way a given repo is set — `git log -1 --format=%B <a squash merge>` shows it directly, and beats reasoning about settings pages.

And verifying once feels like verifying. Running the check for the PR body produces a real sense of having established the fact, which is what stops you checking the other place it appears. The verification is genuine; the coverage is not.

Note the shape is the same as `ardi`'s "an instruction's own suggested code is not exempt", one artifact over: content inherited from a planning document does not feel authored, so the checks you apply to your own claims do not fire on it.

- **Do:** re-run the check when a figure moves from an issue into a commit message, even having verified it once for the PR body.
- **Do:** read `git log -1 --format=%B` before pushing, against the same source the body's claims came from — a commit message is not greppable from the working tree once written.
- **Don't:** copy a count, version, or path out of the tracking issue on the strength of having written that issue.
- **Don't:** treat "permanent in history" as settled while the PR is unmerged — `git commit --amend` still works, and is usually worth a fresh CI round against a wrong figure reaching `main`.

A corollary for checking any of this in a semantic-line-break corpus: a single-line `grep` returns false negatives on your own prose. The instruments above and elsewhere in this file assume you can search for a phrase you wrote. In a corpus that mandates one clause per line, a phrase of any length routinely spans a newline, so `grep 'flat statement of intent'` reports zero against a file that plainly contains it. The failure direction is the dangerous one: a missing-content check that answers "absent" when the content is present reads as a merge having dropped your work, which invites re-doing something already done. Normalize whitespace before matching — read the file, collapse `\s+` to a single space, then search — rather than trusting a line-oriented tool against deliberately broken lines.

Inline markup breaks the same search, and that variant aims the false negative at someone else's work rather than your own. Whitespace is the obvious thing a line-oriented tool gets wrong, so the fix above normalizes it. Markup is the one nobody normalizes, because the two strings *look* identical: a rule titled `Run a local session in an isolated` `` `git worktree` `` `by DEFAULT` is quoted in a citation as "Run a local session in an isolated git worktree by DEFAULT", since prose quoting a title drops its code spans. Grep the quoted form and the definition does not match; only the citation does.

The consequence is worse than the line-break case, and in a specific way. There a false negative says your own merged work is missing, so you re-do something already done. Here it says the **cited** thing is missing, which reads as a dangling citation — and the prescribed response to a dangling citation is to file an issue. So the wrong search does not merely waste effort, it puts a false claim about the corpus into the tracker, against a citation that resolves. A result of exactly one hit, in the citing file, is the tell: a genuinely dangling citation and a formatting mismatch produce the same count, and only reading the hit distinguishes them.

Normalize backticks along with whitespace, or search a distinctive unformatted fragment rather than the whole title.

- **Do:** account for inline markup as well as whitespace before concluding a quoted phrase is absent — see the next block for which side to normalize.
- **Do:** read the single hit when a search for a citation's target returns only the citation itself.
- **Don't:** file a dangling-citation issue while the only evidence is a literal grep that found nothing but the citation — that is the search failing, until a normalized one agrees.

Apply whatever normalization you choose to the search term as well as to the text, or the fix produces a third false negative of its own. Both cases above are answered by transforming the haystack — collapse whitespace, strip backticks — and that framing invites transforming only the haystack, since the needle is the string you already know. But a normalizer is a function, and testing `f(text)` against a raw needle compares two different alphabets. Strip `_` to catch `*emphasis*` and a snake_case identifier stops matching itself: `SH_WORD_SPLIT` becomes `SH WORD SPLIT` in the file while your pattern still carries the underscores, so a term that is present reports absent.

This failure gets *more* likely as the normalizer gets better, which is the part worth naming. Every character class added to catch another markup form is another class that occurs inside real identifiers, so the enumerate-what-to-strip approach converges on breaking the searches it was extended to fix. Enumerating is the wrong shape, not merely an incomplete list.

Running the same function over both sides dissolves the question, whatever the function is:

```
norm = lambda s: re.sub(r"[`*_\s]+", " ", s)
norm(needle) in norm(haystack)
```

- **Do:** normalize the needle with the identical function applied to the text, so the comparison is between two transformed strings.
- **Do:** re-test any earlier absent verdict after extending a normalizer, since the extension can break a term the previous version matched.
- **Don't:** enumerate which markup to strip and treat that list as the fix.
- **Don't:** test a raw search term against normalized text, however plain the term looks.

A flagged item that came in via a `main`-sync merge, not your own diff, is still a Defer — just one where the follow-up is fixing it on `main` directly, not filing a per-PR issue. This is not the ARD skill's "Acknowledge" disposition: `skills/ard/SKILL.md` reserves Acknowledge for praise or a no-ask observation, and explicitly warns against stretching it to dodge a real finding — a redundant config line a reviewer flags is a real finding with an implied fix request, so it needs a real disposition, not a label that means "no change requested." When a reviewer flags something (a redundant config line, a stale pattern) inside a file your branch only touches because you merged `main` in to resolve a conflict, check provenance before fixing it: `git log`/`git blame` the flagged line, or just compare against `origin/main`'s current content. If it's identical to `main`, "fixing" it on your branch alone doesn't fix anything — it just makes your branch disagree with `main` on unrelated content the next person to touch that file will have to reconcile again. Reply agreeing the finding is correct but out of scope for this PR, and leave it for whoever owns that file's actual content to fix on `main` directly — no follow-up issue needed, since the fix target is `main` itself, not this PR's own change.

This generalizes to a skill's own inline restatement of a fragment it links to. A `SKILL.md` that links a backing `shared/` fragment for the full detail often *also* restates the fragment's approach or word list inline (in its `description` field, or a short procedure-step summary) so a reader doesn't have to open the linked file. Fixing a bug in the fragment doesn't automatically fix these inline restatements — they're a second, independent copy of the same claim, and a review round after the fragment fix can catch them going stale exactly like a CHANGELOG paraphrase does. Grep the whole PR diff for the fixed phrase/word-list, not just the fragment file, before considering a fragment fix complete.

A bot that re-raises an item as "not addressed" may simply not have seen your reply — check the timestamps before treating it as an impasse. An automated reviewer gathers the PR's comments once, when its run starts. A rebuttal posted after that

snapshot is invisible to it, so the next round reports the item as still open and unaddressed even though a substantive reply is sitting in the thread. The tell is a re-raise that repeats the original finding verbatim and speaks only to whether the *code* changed, without engaging any argument you made. Before escalating, compare your reply's timestamp against the review run's `started_at` (`gh run view <id> --json startedAt`, or the `started_at` field each run carries in `get_check_runs` when `gh` is absent): if the reply landed after the run began, it is a stale re-raise, not a genuine disagreement. Reply once pointing at the earlier rebuttal (link it directly — the next run will see it), and don't count that round toward the rebuttal-didn't-convince-them test in `fully-clean.md`.

The ordering fix is cheap: when a round is Rebut-only, post the rebuttal **before** anything that triggers the next review (a push, an `@claude` mention), so it is in the snapshot the next run reads. When a round mixes Address and Rebut, post the rebuttals first and push the code second, for the same reason.

Reply-first collides with citing the fix's SHA, and the way out is to commit between them rather than to pick one. The rule above is easy to agree with and still lose, because on a mixed round the reply you want to write says "Addressed in `<sha>`" — and that SHA does not exist until you have committed. So the two instructions read as mutually exclusive: reply first and you have no SHA to cite, push first and the reply misses the next review's snapshot. Pushing first wins that standoff by default, since it is the half that *unblocks* the sentence you were trying to write.

The conflict is only apparent, because committing and pushing are separate steps and only the push triggers review:

1. **Commit** the round's fixes. The SHA now exists and is stable.
2. **Reply** on each thread, citing that SHA.
3. **Push.** The next review's snapshot already contains the replies.

A commit that is never pushed is invisible to CI and to the reviewer, so step 2 is citing something real but not yet reachable — which is fine for a few seconds, and is exactly the window step 3 closes. Note the one thing this does *not* license: the SHA you cite must come from `git rev-parse HEAD` or `git log`, never from recollection, per the PR-body bullet in ardi.

- **Do:** commit, reply citing the committed SHA, then push — in that order.
- **Don't:** treat "I need the SHA for the reply" as a reason to push before replying; that is the ordering the bullet above exists to prevent.

A finding can be right while its `suggestion` block is wrong — verify the suggested literal before applying it. A GitHub ```` ```suggestion ```` block is one-click-appliable, which is exactly what makes an unverified one dangerous: the surrounding prose argues for a change you agree with, so the concrete replacement rides in on that agreement without being checked itself. Treat any file path, version, flag, or command inside a suggestion as a claim to verify,

not as text to accept — the same standard `fact-check-prose` applies to the diff. Accepting a bad literal is worse than ignoring the finding, because it publishes a specific wrong value under the reviewer's apparent authority. When the suggestion is wrong but its point stands, fix the underlying issue your own way and say in the reply why the suggested form was set aside — silently deviating reads as having missed it.

The same check applies to a fix a reviewer describes in prose rather than in a `suggestion` block, and the sharpest test is the reviewer's own example. A finding that ships a concrete repro case has handed you a test fixture: run the proposed fix against that very case before adopting it. A reviewer reasoning about a fix in the abstract can propose one that is directionally right and still insufficient – it closes the failure mode they named while leaving the case they cited broken – and adopting it verbatim converts their partial diagnosis into your shipped bug, with the review thread reading as though the item were settled. When the proposed fix falls short, prefer eliminating the failure mode outright over layering another patch onto it, and post the evidence (the fix applied to their example, and what it still produces) rather than just asserting it was insufficient.

A reviewer's corrected citation is another factual claim, so verify the replacement before adopting it. The finding can be right: the citation in the PR can name the wrong source. That does not make the reviewer's proposed source right. A replacement issue or PR number is a fresh provenance claim, and it needs the same check as the original citation. For text provenance, prefer history over word association: `git log -S "<exact line>" -- <file>` asks which commit introduced the line, while matching a word in another PR plus a nearby merge time only builds a story. Keep the review's conclusion when it is right, but set aside the replacement when the evidence points elsewhere, and say which query decided it.

- **Do:** verify a proposed replacement citation with the source's own history before editing the PR to use it.
- **Do:** use `git log -S "<exact line>" -- <file>` or an equivalent provenance query when the question is which PR introduced text.
- **Don't:** adopt a reviewer's corrected issue or PR number because the original was wrong.
- **Don't:** use word overlap and same-day timing as a substitute for source history.

The highest-yield version of that check: when a comment names an edge case in its own prose and also supplies a fix, run the fix against that edge case. The bullets above test a suggestion against the code, or against a repro the reviewer provided. This tests it against the reviewer's *other paragraph*, and it is the cheapest of them, because the hazard has already been identified for you — the work left is only to check whether the proposed code handles it.

Nothing forces the two halves to agree. A comment's prose and its suggestion are drafted separately, and a reviewer who spots an edge case while reasoning about the problem does not necessarily carry it into the snippet. So a comment can read as unusually thorough — it anticipated a failure mode you had not — while shipping a fix that falls into exactly it. That thoroughness is what makes the suggestion persuasive, which is the trap.

Applying it is worse than ignoring the whole finding. The prose half was right, so the reviewer's authority is real; the snippet then lands under that authority carrying a defect the same comment already described, and the thread reads as settled. Worse still when the defect is one your own corpus documents, since the review has now talked you out of a standing rule.

Keep the finding and reject the snippet. Fix it your own way, quote the edge case back, and say plainly why the suggested form was set aside — silently deviating from a `suggestion` block reads as having missed it.

- **Do:** check a suggested fix against every failure mode the same comment names, before checking anything else about it.
- **Do:** name the reviewer's own caveat in the reply, so the rebuttal rests on their evidence rather than on your say-so.
- **Don't:** let a comment's demonstrated thoroughness transfer to its snippet — they are separate claims.
- **Don't:** discard a finding because its fix is wrong; the half that named the hazard usually still stands.

A quieter variant: the suggestion introduces no defect at all, it restates the line above it — so applying it deletes coverage while reading as hardening. Every bullet above is about a snippet that would break something, so the reviewer's authority is the trap and skepticism is the defence. Here nothing breaks. The tests still pass, the diff looks like a robustness improvement, and the comment is *correct about the problem.* What is lost is the only assertion covering a different property, replaced by a second copy of one already present a line earlier — so a two-assertion test becomes a one-assertion test that still looks like two.

The reason it survives review is that the surviving copy passes, which is indistinguishable from the fix working. So the usual after-the-fact check — run the tests — cannot detect it, and neither can CI. It is a `challenge-redundant-content` finding arriving from the reviewer, which is exactly the direction that makes deferring feel appropriate.

Watch for the comment citing the neighbour as *support*: "the check on the line above is already load-bearing for this claim" is the argument against the replacement, and it reads as an argument for it. Same structure as the edge-case bullet above — prose and snippet drafted separately, disagreeing — one artifact over.

Compare a suggested predicate against its **neighbours**, not only against the code it replaces, and evaluate both on real input rather than reasoning about them; one command decides it, per `algorithmatize-checks`. Then prefer removing whatever made the original fragile over swapping one fragile sentinel for another.

- **Do:** evaluate the suggested predicate and its neighbours on real input, and keep the finding while rejecting the snippet when they coincide.
- **Do:** fix the underlying coupling instead, and say in the reply why the suggested form was set aside.

- **Don't:** accept a `suggestion` block that restates an adjacent check — passing tests afterward prove nothing, since the survivor passes for both.
- **Don't:** read a reviewer's own "the line above already covers this" as support for their replacement.

A finding can be right, and its fix adequate, while the *reason* it supplies is too weak to ship — and in a corpus of rules, the reason is the deliverable. The bullets above all test whether the suggested fix *works*: against the code, against the reviewer's repro, against an edge case their own prose named. This one assumes it works, and asks whether the justification handed to you still holds when someone leans on it.

That distinction is invisible in code and decisive in a rule. A patch is judged by its behaviour, so a correct patch with a shaky rationale is merely under-commented. A `shared/` fragment is judged entirely by whether its reason forecloses the workarounds, so adopting a weaker reason ships a rule the next reader can talk themselves around — while the thread records the item as settled.

The tell is a suggestion that explains *why* something is forbidden in a single phrase, where the primary source carries a stronger provision. So ask what the strongest *available* reason is, rather than whether the offered one is defensible, and name the workaround the weaker reason would have licensed — that is what makes the choice checkable rather than a matter of taste.

- **Do:** read the primary source for the strongest reason before adopting a suggested rationale, even when the suggestion's conclusion is right.
- **Do:** say in the reply which reason you took and why the offered one was set aside, since deviating from a `suggestion` block silently reads as having missed it.
- **Don't:** accept a defensible-sounding mechanism because the conclusion it supports is correct.
- **Don't:** treat this as grounds to reject the finding — the conclusion usually stands, and only its reason needs strengthening.

And the mirror case: a finding can be wrong on its stated grounds while still pointing at something real. The bullets above check the reviewer's *fix*; this one checks their *premise*. A confidently reasoned factual claim – this pattern is valid, that value is in range, this call is safe – invites one of two lazy responses: accept it because it sounds authoritative, or dismiss the whole item once you notice the claim is false. Both lose information, because a reviewer usually arrives at a wrong premise while looking at something that genuinely bothered them.

So reproduce the claim before answering it, and answer the concern separately from the premise. When the premise turns out to be false, say so with the command and its output rather than by assertion, and then address what prompted it anyway – a reader who tested your example and got a different result has a real problem even if their explanation of it was wrong. Expect the corrected mechanism to be more useful than the original text: a premise worth disputing usually sits on something you had not fully explained.

A third direction, which evades the verification reflex rather than lacking a rule: agreeing with a finding and then escalating it. The bullets above check the reviewer's *fix*, and the one above checks their *premise*. Both assume you are deciding whether the finding is right. This is the case where it is right, and correctly scoped, and you tell the reviewer it understated the problem.

The **obligation** is not new, and claiming otherwise would overstate this entry. `metacognitive-monitoring` already requires verifying a finding's particulars before restating them as fact, and its **scope** claim type already governs an assertion of your own about how wide a defect is, telling you to check the population rather than the sample that came to mind. An escalation is nothing but a new particular of exactly that kind: a wider scope, a bigger count, one more failing case. So it lands squarely in the class that fragment names as least dependable, and it does so where the reviewer's credibility will carry it.

What is new is the **trigger**. Both of those rules fire on an act you recognize as asserting something, and agreeing does not present as one. Rebutting is adversarial and prompts you to verify. Extending is agreement wearing extra diligence, and agreement is not a thing anyone verifies. So the rule is already there and nothing calls it, which is how the escalation ships under the reviewer's authority with less scrutiny than a rebuttal would have got.

Hold an escalation to the standard a rebuttal gets. Measure with an instrument covering the whole scope your escalation claims, not merely the narrower scope the finding covered, and say which instrument that was. Do not read the finding's narrowness as a bound on the reviewer's instrument. A reviewer can inspect a whole field set and report only the member that is broken, so a one-field finding can rest on a five-field probe. The instrument you need may therefore be the reviewer's own, run without whatever narrowed your view of it, rather than a new and wider one. When the escalation turns out to be wrong, correct it on the thread that carried it, not only in a later round's summary. A reader who saw "it is worse than you reported" has no other way to learn that it was not.

- **Do:** verify an escalation against the full scope it claims, which is wider than the scope the finding reported, and which the finding's own instrument may already cover.
- **Do:** post the correction to the thread that carried the escalation.
- **Don't:** treat agreeing-and-extending as exempt from the checks a rebuttal gets, since agreement suppresses the reflex that disagreement triggers.
- **Don't:** report a finding as understated on a measurement you have not shown covers the whole field set.

When a finding cites a source, read the cited source before reproducing anything – it is the cheaper instrument, and it is the one that can show the finding backwards rather than merely unsupported. The bullet above says to reproduce the claim. That is right, and it is the second thing to do when a citation is on the table, because reproduction tests the *behavior* while the citation tests the *reasoning*, and only the second can catch a finding whose own evidence contradicts it. A citation is also the most persuasive part of a review and the least likely to be checked: a linked changelog entry reads as settled fact, so

the finding inherits authority it never earned, and a one-click `suggestion` block turns that borrowed authority into an applied edit.

Grep the cited document for the mechanism the finding names. One command usually decides it, which makes this an `algorithmatize-checks` case rather than a judgment call, and a fabricated mechanism produces a clean zero-hit result that is hard to argue with. Then quote the entry in the reply rather than paraphrasing the disagreement, and reproduce the behavior as the independent second leg.

Do not stop at winning the point. A finding that misread a source usually did so because the claim it questioned had nothing checkable next to it, so fold the citation into the file itself, per `fully-clean`'s note that a fresh review run re-derives from scratch and will not read the thread.

When a reviewer hedges a finding because it depends on code it cannot see, check whether *you* can see it — the hedge is an invitation, not a verdict. Automated reviewers work from the diff, so a finding that turns on a reusable workflow, a dependency's internals, or another repo's behavior arrives with language like "moderate rather than high confidence", "depends on behavior not visible in this diff", or "worth the author confirming intent". That hedge is a fact about the *reviewer's* visibility, not about how likely the finding is. You frequently have access it lacks: the repo cloned locally, a pinned dependency vendored in, or permission to fetch the source.

Reading it converts a maybe into a settled yes or no, and that changes the disposition. Confirmed, it earns a fix or a precisely-scoped follow-up issue with the mechanism recorded; disproved, it earns a Rebut with evidence instead of a vague "I think this is fine". Either way the next reader is spared re-deriving it. Quote the specific lines you checked, since a follow-up issue that merely repeats the reviewer's hedge is barely more useful than the review comment it came from.

Timestamp the evidence before rebutting a finding with it — during a live incident, a log from twenty minutes ago describes a different system. The bullets above all say to verify a finding rather than accept it, and they assume verification is a fixed target: read the source, run the command, reproduce the case. That assumption quietly fails while something is actively breaking, because the evidence you gather is a *measurement*, and measurements expire. Re-reading an existing CI log feels like verification — it is concrete, it is specific, it is right there — but it only tells you what was true when that job ran.

The tell is a rebuttal whose evidence you did not generate yourself in this turn. A log you fetched, a check-run conclusion you read, a status you were told about: each carries a timestamp, and the question is whether anything could have changed since. When the finding is *about* an outage, a migration, a permission change, or anything else in flight, the answer is almost always yes.

So prefer evidence you can regenerate now over evidence you can only cite. Re-running the failing thing is usually cheap and settles it outright — and in the best case it produces the

cleanest possible proof, two attempts of the same run on the same commit disagreeing, which no amount of reading could have given you. When regenerating is genuinely not possible, say how old the evidence is in the rebuttal itself, so the reader can weigh it.

This matters more than an ordinary wrong rebuttal because of who it lands on. Telling an author their diagnosis is contradicted by the logs is a strong claim that invites them to stop investigating. Getting it wrong can stall a correct fix for the exact bug still breaking everything.

A finding built on a *negative* result – "I searched and it isn't there" – is only as strong as the paths that were searched, and the search scope is the part reviewers state loosest. The bullets above all check a reviewer's positive evidence: the suggested literal, the proposed fix, the cited source. A negative result invites none of that scrutiny, because there is nothing to look up: the claim is that looking up would fail. It also arrives sounding the most settled of any finding – "no file or heading with that title anywhere" reads as exhaustive, and the reader's natural move is to accept it and edit.

So read the search itself rather than the conclusion. Ask which paths were actually covered, and whether the obvious location is among them. One command usually settles it, which makes this an `algorithmatize-checks` case rather than a judgment call – and note the reviewer's own tooling may have failed the way `fail-fast`'s hand-check section describes, matching too narrowly against text that was wrapped or reformatted.

When it turns out the thing does exist, name the gap rather than only the correction: which paths were searched, where it actually lives, and why the two did not overlap. That is what stops the same search being re-run the same way. And check whether the finding still points at something real, per the mirror case above – an unresolvable-looking citation is often a genuinely under-specified one.

- **Do:** ask which paths a negative finding actually searched, and check the obvious location yourself before editing anything.
- **Do:** name the gap when the thing does exist – paths searched versus where it lives – so the same search is not re-run the same way.
- **Don't:** accept "it isn't there anywhere" as settled because it is stated more confidently than a positive finding would be.
- **Don't:** discard the finding once its negative result is disproved – the thing it tripped over is often a real ambiguity.

A note the reviewer declined to raise is still a claim, and so is your refutation of it. Every bullet above checks a finding the reviewer actually **raised** — the suggested literal, the proposed fix, the cited source, the negative result. A note dropped in passing, marked out of scope, or explicitly declined is checked by nobody, precisely because nobody is asking you to act on it.

It can be right, and it can be wrong, and both directions cost something. `ardi`'s "not a finding" section owns the right-and-ignored direction: a reviewer can analyse a real convention violation

correctly and still grade it acceptable, so the part of a review most likely to be skimmed is where a genuine violation sits. This bullet is the other half — what happens once you do go and check.

The trap is that **checking a declined note feels like the end of the verification when it is the start of a second unverified claim.** Overturning something reads as more rigorous than accepting it, so a refutation draws less scrutiny than the note it overturns rather than more. And a refutation is unusually cheap to get wrong here, because the artifact nearest to hand is not evidence about the code: a PR title, a commit subject, or a changelog line describing a refactor is a claim about **intent**, and the code is whatever that intent left behind. "Converted from a position push to a velocity impulse" and a function that computes a velocity and then still writes the position on the next line are entirely compatible, and only one of the two is the code.

So read the function rather than the sentence describing the change to it, and read it at the current tip — a checkout that predates the merge answers a different question, and answers it confidently.

Holding is usually still right, and `efficient-pr-babysitting` already gives one reason: a declined note is not an open item, and a clean verdict standing over it is a stop. Verifying supplies a second and better one, because that rule's argument is about **cost** — a round of CI and re-review spent on something that was never blocking — which argues for holding whether the note is right or wrong. Checking tells you which of those you are in, it is usually one command, and the two compose: verify the note, then hold anyway unless it named a real defect.

- **Do:** verify a declined, out-of-scope, or passing note against the code before either acting on it or writing it off.
- **Do:** hold the change regardless when the note turns out correct but genuinely optional — verifying decides what is true, not what ships.
- **Don't:** treat a PR title, commit subject, or changelog line as evidence about what the code does; each states an intent, and a refactor can keep the very thing it says it replaced.
- **Don't:** let your own refutation past the check you would have applied to the reviewer's finding — it is a fresh claim, and overturning something feels like having verified it.

References